



UNIVERSIDADE ESTADUAL PAULISTA
“JÚLIO DE MESQUITA FILHO”
Instituto de Ciências e Tecnologia - Campus de
Sorocaba - ICTS

UNIVERSIDADE ESTADUAL PAULISTA “JÚLIO DE MESQUITA FILHO” -
FACULDADE DE CIÊNCIAS E TECNOLOGIA, CAMPUS DE SOROCABA

LETÍCIA MEIRELLES NOVAES DE ANDRADE 241270375
REBEKA SANTOS RODRIGUES 231270747

**SISTEMA DE CLASSIFICAÇÃO E ALERTA DE TERREMOTOS COM
ESTRUTURAS DE DADOS**

SOROCABA

2025

LETÍCIA MEIRELLES NOVAES DE ANDRADE - 241270375

REBEKA SANTOS RODRIGUES - 231270747

**SISTEMA DE CLASSIFICAÇÃO E ALERTA DE TERREMOTOS COM
ESTRUTURAS DE DADOS**

Trabalho apresentado no Curso de graduação em Engenharia de Controle e Automação, na disciplina de Estrutura de Dados na Universidade Estadual Paulista “Júlio de Mesquita Filho”, Faculdade de Ciências e Tecnologia, Campus de Sorocaba.

Docente: Dr. Leopoldo André Dutra Lusquino Filho.

SOROCABA

2025

SUMÁRIO

INTRODUÇÃO E OBJETIVO.....	5
1. DATASET GLOBAL EARTHQUAKE (2015-2025).....	6
1.1. Análise do Dataset.....	6
1.1.1. Análise latitude e longitude.....	6
1.1.2. Profundidade.....	8
1.1.3. Impact score.....	9
1.1.4. Magnitude.....	10
1.1.5. Matriz de correlação dos dados numéricos.....	11
1.1.6. Modelo de regressão linear.....	12
1.2. Filtragem de Dados.....	13
2. ESTRUTURA DE DADOS IMPLEMENTADAS.....	13
2.1. Estruturas Clássicas.....	13
2.1.1. Estrutura Lista duplamente encadeada.....	13
2.1.2. Estrutura Árvore AVL.....	13
2.1.3. Estrutura Tabela hash.....	14
2.2. Estrutura Recentes.....	14
2.2.1. Estrutura Skip list.....	14
2.2.2. Estrutura KD-Tree.....	15
2.3. Estrutura Otimizada (Perfect Hash Table).....	15
3. ESPECIFICAÇÃO DO SISTEMA.....	16
3.1. Monitoramento de Terremotos.....	16
3.1.1. Adicionar Novo Registro.....	17
3.1.2. Buscar Registro.....	17
3.1.3. Remover Registro.....	17
3.1.4. Buscas Extras KD-Tree.....	18
3.2. Cálculos Estatísticos Sobre os Dados.....	18
3.3. Previsão de Impact Score com Base na Magnitude.....	19
3.4. Alerta de Terremotos.....	20
3.5. Classificação de Risco Sísmico.....	20
3.6. Executar Benchmarks.....	21
4. RESULTADOS DOS BENCHMARK E DISCUSSÃO.....	23
4.1. Tempo total de inserção (ms).....	23
4.2. Tempo médio de inserção por elemento (us).....	24
4.3. Tempo médio de busca de dados existentes (us).....	24
4.4. Tempo médio de busca de dados inexistentes (us).....	25
4.5. Tempo médio de remoção (us).....	25
4.6. Latência média de operações mistas.....	26

4.7. Uso de memória estimado.....	26
4.8. Taxa de colisões.....	27
4.9. Trade-off.....	27
5. CONCLUSÃO.....	29
REFERÊNCIAS.....	30
APÊNDICE A - TABELAS DE RESULTADOS DOS BENCHMARK.....	31
APÊNDICE B - GRÁFICOS DOS RESULTADOS DOS BENCHMARK.....	54
APÊNDICE C - PROMPT USADO PARA APOIO DE INTELIGÊNCIA ARTIFICIAL.	
68	

INTRODUÇÃO E OBJETIVO

O presente trabalho tem como objetivo comparar o desempenho de diferentes estruturas de dados aplicadas a um dataset real de terremotos globais. Para isso, foi desenvolvido um sistema, utilizando a linguagem C, capaz de realizar operações de inserção, busca e remoção de registros sísmicos com o uso de diversas estruturas de dados. Além dessas operações, o sistema oferece funcionalidades adicionais, como análises estatísticas, previsão de tendências e emissão de alertas com base em critérios pré-definidos.

As estruturas de dados avaliadas incluem listas duplamente encadeadas, árvores AVL, tabelas hash, skip lists, KD-trees e perfect hashing. Todas foram analisadas por meio de benchmarks, considerando critérios como tempo de execução e uso de memória. Os códigos que compõem o sistema foram desenvolvidos com o apoio de ferramentas de inteligência artificial.

O sistema desenvolvido busca atender à necessidade de classificar as regiões mais suscetíveis a eventos sísmicos potencialmente perigosos, contribuindo para a prevenção de desastres, a alocação eficiente de recursos, o planejamento urbano e a preparação para situações de emergência.

A distribuição de tarefas foi feita da seguinte forma: a integrante Letícia Meirelles ficou responsável pela análise estatística, estruturas de dados, otimização da estrutura (perfect hash), menu que integrou as estruturas e benchmark. Já a integrante Rebeka ficou responsável pelas funções de classificação dos países, alerta de terremoto, regressão linear, filtragem dos dados, estruturas de dados, otimização da estrutura (perfect hash).

Obs: As estruturas foram feitas em conjunto, assim como a maior parte do trabalho.

1. DATASET GLOBAL EARTHQUAKE (2015-2025)

O conjunto de dados escolhido, retirado do site kaggle em formato CSV, consiste em registros estruturados relacionados a ocorrências de terremotos, compilados a partir de diversas fontes de monitoramento sísmico e abrange vários países e regiões. contém 16.944 amostras e 9 colunas com as seguintes especificações:

Data – A data específica em que o terremoto ocorreu.

Hora (UTC) – A hora exata do terremoto no Tempo Universal Coordenado (UTC).

Cidade – A cidade ou ponto de referência mais próximo do epicentro do terremoto.

País – O país onde o terremoto ocorreu.

Latitude – A coordenada de latitude do epicentro do terremoto.

Longitude – A coordenada de longitude do epicentro do terremoto.

Magnitude do Terremoto – A força do terremoto na escala Richter.

Profundidade (km) – A profundidade de origem do terremoto, medida em quilômetros.

Pontuação de Impacto/ Impact Score – Uma pontuação numérica que indica a gravidade potencial do terremoto.

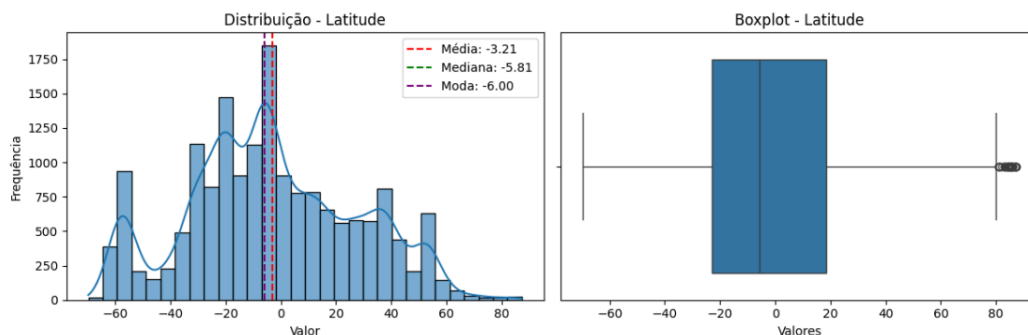
1.1. Análise do Dataset

Antes de começar a construção do sistema, foram realizados cálculos estatísticos para análise do dataset então, a partir disso, decidir os critérios para classificação dos países. Os cálculos e gráficos foram gerados usando a linguagem python, os códigos usados podem ser encontrados no arquivo “*calc_estatistico_filtragem.ipynb*”.

Foram feitas análises de todos os dados numéricos, mas aqui serão abordados os mais relevantes para a construção do sistema.

1.1.1. Análise latitude e longitude

Latitude



Coeficiente de Assimetria: 0.15

Coeficiente de Curtose: -0.51

Desvio Padrão: 30.58

Variância: 935.12

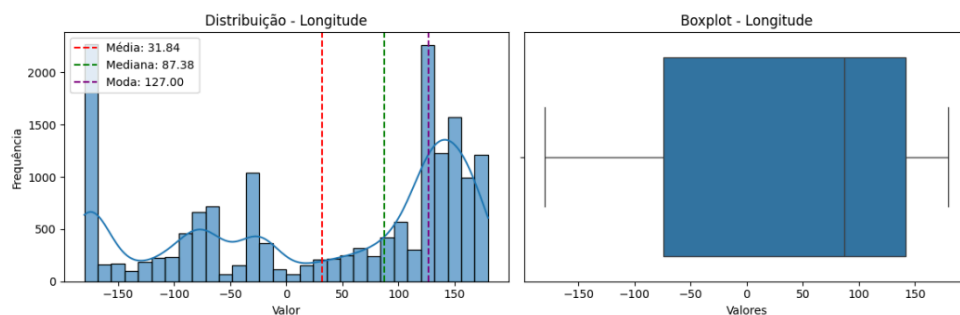
Medidas de Centralidade Calculadas:

Média: -3.207031967359341

Mediana: -5.8123000000000005

Moda: -6.0

Longitude



Coeficiente de Assimetria: -0.49

Coeficiente de Curtose: -1.29

Desvio Padrão: 123.92

Variância: 15356.87

Medidas de Centralidade Calculadas:

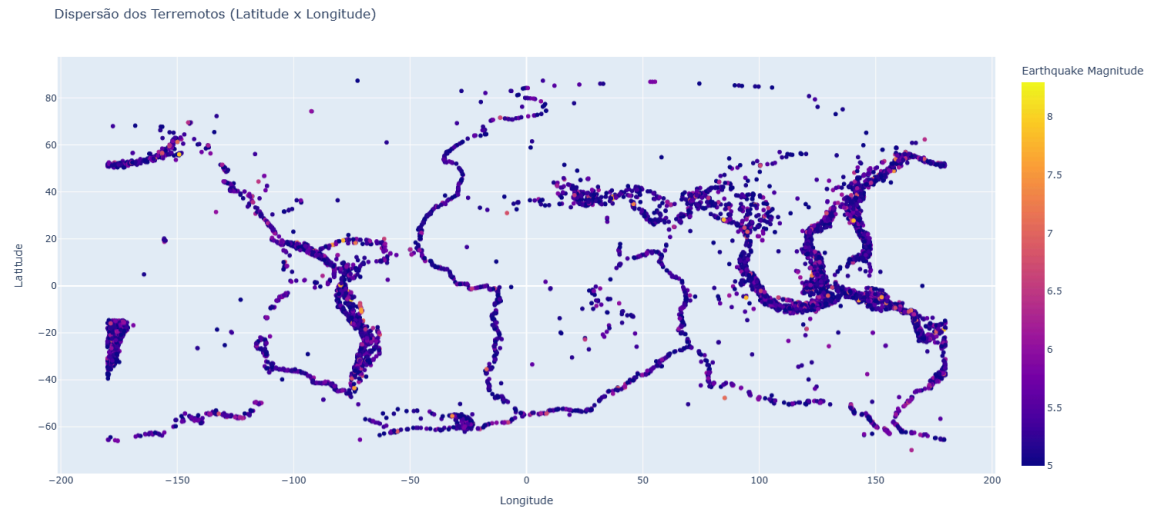
Média: 31.839993782235798

Mediana: 87.3783

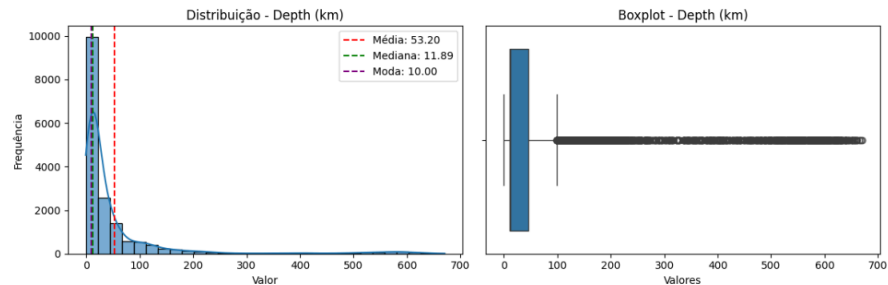
Moda: 127.0

Os dados geográficos mostram predominância de eventos no Hemisfério Sul, com média negativa para a latitude, mas boa dispersão devido à baixa assimetria. Para longitude, observa-se maior concentração em valores mais baixos, possivelmente refletindo a presença de placas tectônicas ou zonas sísmicas mais ativas em longitudes específicas. As curtoses negativas significam menos valores extremos, indicando que a maioria dos terremotos ocorre em regiões geográficas parecidas.

Para análise, também foi feito um gráfico de dispersão dos terremotos (Latitude x Longitude), como mostrado na imagem a seguir, é interessante que a dispersão acaba formando as posições das placas tectônicas.



1.1.2. Profundidade



Coefficiente de Assimetria: 3.87

Coefficiente de Curtose: 15.63

Desvio Padrão: 104.09

Variância: 10834.92

Medidas de Centralidade Calculadas:

Média: 53.202267457956616

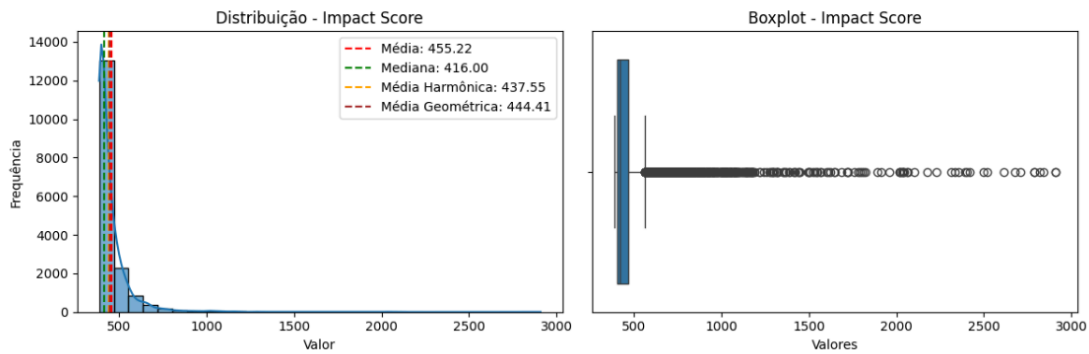
Mediana: 11.8925

Moda: 10.0

A assimetria fortemente positiva indica que a maioria dos terremotos ocorre em profundidades baixas. A curtose extremamente alta, mostra que existem outliers extremos, provavelmente os poucos terremotos muito profundos. Também é possível observar que a

média maior que a mediana, mostra que os poucos valores de terremotos profundos aumentam bastante a média

1.1.3. Impact score



Coefficiente de Assimetria: 7.69

Coefficiente de Curtose: 91.82

Desvio Padrão: 137.23

Variância: 18832.62

Medidas de Centralidade Calculadas:

Média: 455.2163597733711

Mediana: 416.0

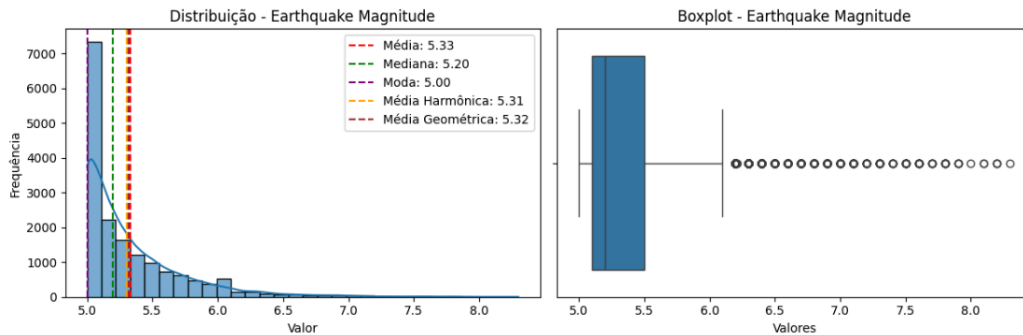
Moda: 385

Média Harmônica: 437.5522853638125

Média Geométrica: 444.41433593613795

Assimetria e curtose altas, indicam que a maioria dos eventos tem impacto menor, mas existem alguns eventos catastróficos, que são outliers.

1.1.4. Magnitude



Coeficiente de Assimetria: 2.14

Coeficiente de Curtose: 6.26

Desvio Padrão: 0.40

Variância: 0.16

Medidas de Centralidade Calculadas:

Média: 5.333485009442871

Mediana: 5.2

Moda: 5.0

Média Harmônica: 5.306723941551641

Média Geométrica: 5.319517308721593

A assimetria positiva alta, indica que existem muito mais eventos de baixa magnitude, ou seja, os eventos intensos são raros. A alta curtose, mostra um pico muito acentuado, ou seja, uma concentração do valor da magnitude perto da média, mas também caudas mais pesadas, o que significa que algumas magnitudes extremas (muito altas) ocorrem com mais frequência do que numa distribuição normal.

Outra análise estatística feita com a magnitude e também com o impact score, foi separar os intervalos de magnitude, de 0 a 5, de 5 a 6 e de maior que 6, e ver a média do impact score dentro de cada intervalo, além de ver qual foi o impact score mínimo e máximo, a seguir o resultado:

Magnitude de 0 a 5:

Média do Impact Score: 387.7479270315091

Impact Score Mínimo: 385

Impact Score Máximo: 1095

Magnitude de 5 a 6:

Média do Impact Score: 451.20345242184817

Impact Score Mínimo: 400

Impact Score Máximo: 2392

Magnitude de 6 a inf:

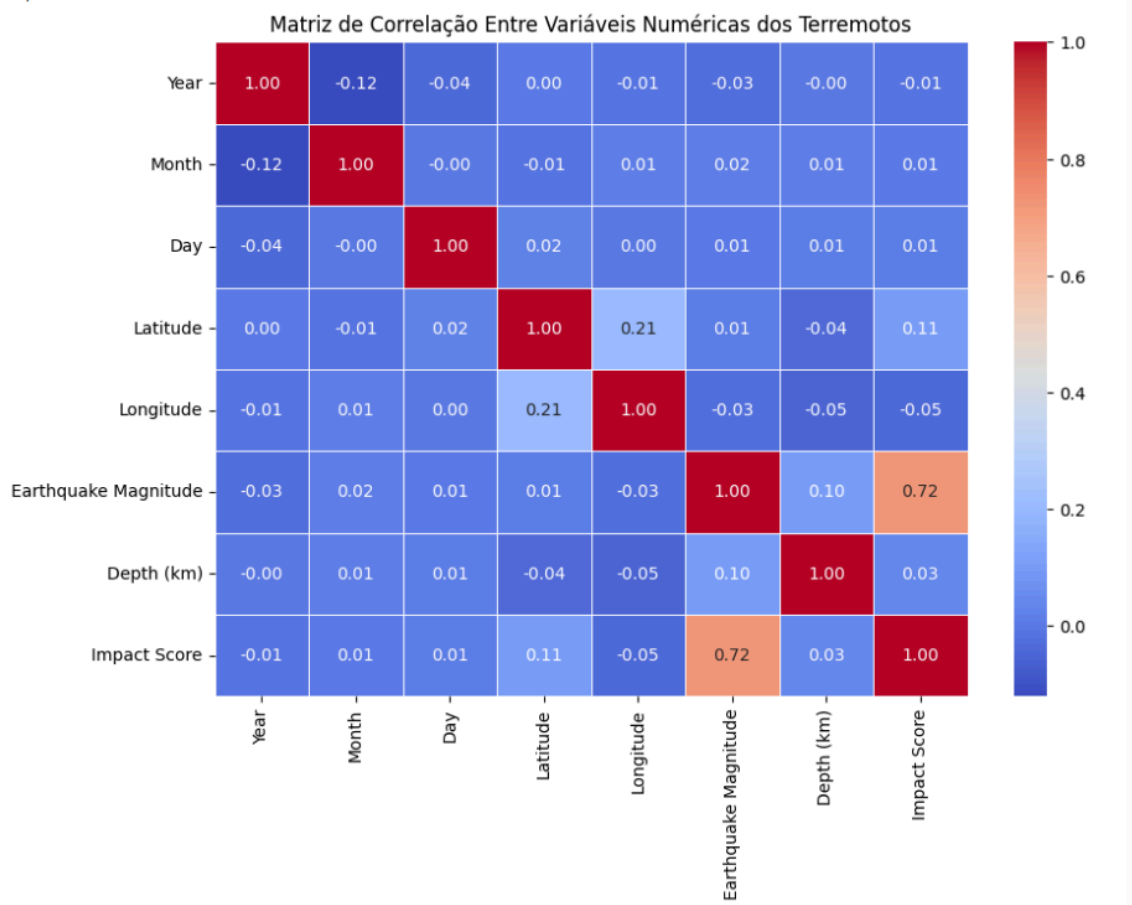
Média do Impact Score: 762.4550509731232

Impact Score Mínimo: 426

Impact Score Máximo: 2910

Esses resultados foram usados para decidir a classificação do nível de risco dos países, que é explicado na seção 3.5 deste relatório

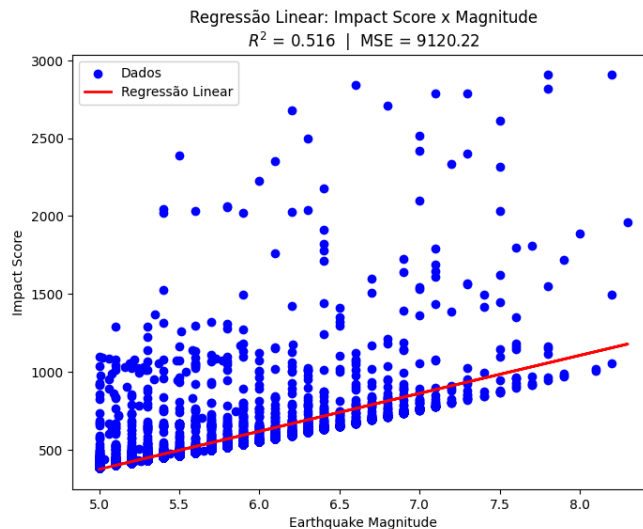
1.1.5. Matriz de correlação dos dados numéricos



A correlação mais forte pela matriz com o valor de 0.72, é entre a magnitude e o impact score, indicando que terremotos com maior magnitude tendem a causar maiores impactos.

1.1.6. Modelo de regressão linear

Por causa do resultado da matriz de correlação, as variáveis impact score e magnitude foram escolhidas para montar um modelo de regressão linear, obtendo os



seguintes resultados:

Equação: $\text{Impact Score} = 243.7911 * \text{Magnitude} + -845.0397$

Coefficiente de determinação (R^2): 0.5157

Mean Squared Error (MSE): 9120.2211

A equação do modelo obtido, diz que para cada ponto adicional na magnitude do terremoto, o impact score aumenta em média 243.79 pontos.

O coeficiente de determinação R^2 mostra que aproximadamente 51,57% da variação do impact score pode ser explicado pela magnitude do terremoto, pela quantidade de amostras o modelo é estatisticamente relevante, ou seja, é um modelo bom mas sugere que o impacto também é explicado por outras variáveis.

O erro quadrático médio (MSE) fez a medida da diferença média ao quadrado entre os valores reais do impact score e os previstos pelo modelo, o valor apresentado foi 9120.22. O erro absoluto pode ser aceitável para algumas aplicações, mas grande para outras, especialmente para prever valores nos extremos mais baixos da escala de Impact Score, onde o erro percentual se torna maior.

1.2. Filtragem de Dados

No sistema foi implementado uma função para depois que o dataset for lido, sejam removidos os registros inválidos (com campos obrigatórios vazios, no caso todos os campos menos o horário).

2. ESTRUTURA DE DADOS IMPLEMENTADAS

2.1. Estruturas Clássicas

2.1.1. Estrutura Lista duplamente encadeada

A lista duplamente encadeada é uma estrutura de dados linear na qual cada elemento (ou nó) possui dois ponteiros: um para o elemento anterior e outro para o próximo. Isso permite uma navegação bidirecional, tanto para frente quanto para trás, facilitando buscas, inserções e remoções em qualquer posição da lista de forma eficiente. Como cada nó conhece seus vizinhos, é possível realizar inserções e remoções em tempo constante ($O(1)$) desde que o nó de referência já seja conhecido.

A lista duplamente encadeada está implementada no arquivo *lista_dupla_ligada.cpp*, onde cada nó da lista armazena um registro completo de terremoto e ponteiros para os nós anterior e seguinte. Os registros são inseridos sequencialmente ao final da lista por meio da função *insert_record*, mantendo a ordem cronológica de chegada dos dados.

2.1.2. Estrutura Árvore AVL

A árvore AVL é uma estrutura de dados do tipo árvore binária de busca auto-balanceada. Cada nó possui um fator de balanceamento, a diferença entre as alturas das subárvores esquerda e direita, e a árvore se mantém equilibrada a cada inserção ou remoção, garantindo que a profundidade permaneça proporcional a $\log(n)$, mesmo com grandes volumes de dados. Isso assegura que as operações de busca, inserção e remoção tenham desempenho estável e eficiente, com complexidade $O(\log n)$, tornando essa estrutura especialmente útil quando é necessário acesso ordenado e rápido aos dados.

No sistema de classificação e alerta de terremotos, a árvore AVL está implementada no arquivo *avl_tree.cpp*, cada registro é indexado por uma chave composta, gerada a partir da data, horário, cidade e país, permitindo identificação única e organização lexicográfica dos eventos. A função *insert_record* insere os dados mantendo o balanceamento da árvore, enquanto operações como *search_record_by_composite_key*, *remove_record*, *list_by_year* e *list_by_min_magnitude* utilizam percursos ou rotações internas para garantir desempenho mesmo em cenários de uso intensivo.

2.1.3. Estrutura Tabela hash

A Tabela Hash é uma estrutura de dados que permite armazenar e acessar informações com alta eficiência, usando o conceito de mapeamento de chave para índice por meio de uma função hash. Cada elemento é inserido em um "bucket" (ou lista encadeada) baseado em um índice calculado a partir da chave. Essa abordagem oferece tempo de acesso, inserção e remoção médio constante ($O(1)$), desde que a função hash seja bem distribuída e o fator de carga da tabela seja controlado. A colisão de chaves, quando diferentes registros são atribuídos ao mesmo índice, é resolvida por encadeamento separado, permitindo que múltiplos registros coexistam em um mesmo bucket.

No sistema a Tabela Hash está implementada no arquivo *hash_table.cpp*. Cada registro sísmico é indexado por uma chave composta, gerada a partir da data, hora e cidade do evento. A estrutura utiliza um vetor de listas (*std::vector<std::list<HashNode>>*) para armazenar os dados internamente, e a função *generate_key* cria as chaves compostas. Operações como *insert_record*, *search_record* e *remove_record* permitem controlar rapidamente os registros por meio dessas chaves. A função *rehash* é utilizada automaticamente para redimensionar a tabela quando o número de elementos ultrapassa um determinado fator de carga, garantindo desempenho consistente.

2.2. Estrutura Recentes

2.2.1. Estrutura Skip list

A Skip List é uma estrutura de dados probabilística que combina o desempenho de árvores balanceadas com a simplicidade de listas encadeadas. Ela é composta por múltiplos níveis de nós, onde o nível 0 representa uma lista encadeada comum e os níveis superiores contém "atalhos" (saltos) que permitem pular blocos de nós para acelerar buscas. Graças à sua organização hierárquica e à atribuição aleatória dos níveis de cada nó, a Skip List atinge complexidade média de $O(\log n)$ para inserção, remoção e busca, mantendo uma implementação conceitualmente mais simples do que árvores balanceadas como AVL.

No sistema, a Skip List está implementada no arquivo *skip_list.cpp*. Cada nó armazena um registro sísmico (*EarthquakeRecord*) e é indexado por uma chave composta baseada em data, hora e cidade. O nó especial chamado header marca o início da estrutura e serve como ponto de partida para todas as operações. A função *insert_record* utiliza um gerador de nível aleatório (*random_level*) para definir quantos níveis o novo nó ocupará, enquanto as operações de busca (*search_record*) e remoção (*remove_record*) exploram os

múltiplos níveis para localizar rapidamente o nó correspondente. Por sua eficiência e estrutura adaptável, a Skip List se mostra vantajosa para cenários que exigem ordenação dinâmica com acessos frequentes.

2.2.2. Estrutura KD-Tree

A KD-Tree (K-Dimensional Tree) é uma estrutura de dados especializada para armazenar e buscar informações em espaços multidimensionais. Cada nó da árvore representa um ponto no espaço, e a árvore divide recursivamente o espaço ao longo de diferentes eixos em cada nível. Isso permite realizar buscas espaciais como “pontos mais próximos” (nearest neighbor), “pontos dentro de uma área geográfica” (range search), ou “os k pontos mais próximos” (k-NN) de forma eficiente, com complexidade média $O(\log n)$ para inserção e busca.

No sistema, a KD-Tree está implementada no arquivo `kd_tree.cpp` e utiliza dois eixos (latitude e longitude) para armazenar registros sísmicos. Cada registro é inserido com verificação e conversão de suas coordenadas usando o helper `KDTreeHelpers::try_string_to_double`. A função *insert_record* organiza os dados geograficamente, permitindo que a árvore se auto equilibre ao longo dos dois eixos. As buscas por região (*search_by_range*) e por proximidade (*search_nearest_neighbor* e *search_k_nearest_neighbors*) exploram esse particionamento espacial para responder rapidamente a consultas de localização. Por sua capacidade de lidar com dados geográficos de forma hierárquica e eficiente, a KD-Tree é especialmente adequada para esse sistema. Obs: Alguns dados não conseguiram ser inseridos na KD-Tree devido à um erro na leitura da latitude do dataset (apesar do dataset não apresentar erros).

2.3. Estrutura Otimizada (Perfect Hash Table)

A estrutura escolhida para ser melhorada foi a Tabela Hash, por ser boa em busca, que é umas das operações mais frequentes do nosso sistema, então utilizá-la é um ganho significativo

A Perfect Hash Table é uma estrutura de dados de busca baseada em hashing que busca eliminar colisões completamente, ou seja, garantir que cada chave seja mapeada a uma única posição na tabela, sem a necessidade de sondagem ou encadeamento. Isso é especialmente útil quando o conjunto de dados é estático (não muda com frequência).

O sistema em questão implementa uma “perfect hash” utilizando o algoritmo conhecido como FKS (Fredman, Komlós e Szemerédi), que adota uma abordagem em dois

níveis: uma tabela de hash primária e várias tabelas de hash secundárias. A chave utilizada para indexar os registros sísmicos é composta pela concatenação de três atributos: data, hora e cidade. Isso assegura a unicidade de cada entrada, mesmo em casos de múltiplos eventos ocorrendo na mesma cidade em um mesmo dia.

3. ESPECIFICAÇÃO DO SISTEMA

O sistema tem como objetivo classificar os terremotos já existentes e gerar alertas de terremotos instantaneamente. Além disso, ele permite que o usuário escolha uma das 5 estruturas do sistema para realizar o monitoramento de terremotos, também permite obter cálculos estatísticos, previsão de tendência, cadastro de e-mails para o envio dos alertas e execução de benchmarks para análise das estruturas.

```
===== MENU PRINCIPAL =====
1. Selecionar Estrutura de Dados para Monitoramento
2. Alerta de Terremoto
3. Calculos Estatisticos sobre os Dados
4. Previsao de Impact Score com base na magnitude
5. Ver classificação de risco
6. Executar Benchmarks
0. Sair do Programa
Sua escolha: █
```

3.1. Monitoramento de Terremotos

O monitoramento de terremotos consiste em um submenu, onde é possível inserir o registro de um novo terremoto, realizar busca por país, magnitude exata e mínima, classificação e por cidade e magnitude, e remover um registro de duas maneiras, adicionando todas as informações do registro para remover, ou remover o último registro buscado. Para isso, primeiro o usuário escolhe qual estrutura deseja utilizar. O header `operations_menu.h` implementa esse submenu interativo para o usuário gerenciar dados de terremotos. Ele é genérico e opera com as estruturas de dados (Lista Duplamente Ligada, Árvore AVL, KD-Tree, Tabela Hash, Skip List, Perfect Hash Table). A função (`DataStructureType`) é definida como um template, e o tipo exato da estrutura de dados (`data_structure`) é especificado quando a função é chamada. Isso permite que o mesmo código de menu funcione para diferentes implementações de estruturas. Para operações como inserir (`insert_record`), remover (`remove_record`), listar (`list_all_records`, etc.), a função chama os métodos correspondentes do objeto `data_structure` que foi passado como parâmetro.

Para buscas específicas da estrutura (como `search_by_range` na KD-Tree ou `search_record` por chave na HashTable), ela também chama os métodos especializados da `data_structure`.


```
--- Selecionar Estrutura de Dados ---  
1. Lista Duplamente Ligada  
2. Arvore AVL  
3. Tabela Hash  
4. KD-Tree  
5. Skip-List  
6. Tabela Hash Perfeita  
0. Voltar ao Menu Principal  
Escolha a estrutura: █
```

```
--- Menu Monitorar Terremotos (Usando Lista Duplamente Ligada) ---  
1. Adicionar Novo Registro  
2. Remover Registro (Identificacao Manual)  
3. Buscar Registro (Por Atributos)  
4. Listar Registros  
0. Voltar ao Menu Anterior  
Sua escolha: █
```

3.1.1. Adicionar Novo Registro.

O menu é auto explicativo, todas as estruturas seguem o mesmo padrão, salvo algumas especificações a mais que algumas estruturas permitem. Para adicionar um novo registro basta digitar todas as informações, data que aconteceu, hora, cidade, país, magnitude, impact score, latitude, longitude e profundidade.

3.1.2. Buscar Registro

A busca pode ser realizada por país, magnitude exata, pela data, pelo nível de risco do país, e digitando a cidade e magnitude.

Por exemplo, se a opção escolhida for a magnitude exata, quem está usando o sistema digita a magnitude e é listado todos os registros de terremoto com a magnitude escolhida.

3.1.3. Remover Registro

Para remover o registro, tem duas opções, a primeira é por identificação manual que permite remover um único registro colocando todas as suas exatas informações, ou selecionando a opção “Remover Registro(s) da Última Busca”, que aparece depois de realizar alguma busca, removendo todos os registros listados na última busca

3.1.4. Buscas Extras KD-Tree

No geral, os menus de cada estrutura são iguais, mas a KD-Tree foi projetada especialmente para trabalhar com dados multidimensionais, como coordenadas geográficas (latitude e longitude), então para explorar suas especificações, o sistema permite outras três opções de busca com essa estrutura.

Busca por range geográfico: o usuário digita os valores mínimo e máximo de latitude e de longitude, e o sistema lista todos os registros de terremoto que ocorreram dentro dessa área geográfica.

Buscar vizinho mais próximo: digitando as coordenadas específicas de latitude e longitude, o sistema encontra e exibe o registro de terremoto mais próximo.

Buscar K vizinhos mais próximos: semelhante a anterior, mas nessa tem a adição do usuário escolher a quantidade de registros mais próximos da coordenada geográfica que quer obter.

3.2. Cálculos Estatísticos Sobre os Dados

Uma das funções que o sistema permite é a *statistic.cpp*, que realiza cálculos estatísticos entre média, mediana, desvio padrão e variância, para isso o usuário escolhe entre uma das seguintes variáveis: latitude, longitude, magnitude, profundidade, impact score, ano/mês/dia do terremoto.

```

=== Análises Estatísticas ===
Escolha uma variável para análise:
1. Latitude
2. Longitude
3. Magnitude
4. Profundidade
5. Impact Score
6. Ano do Terremoto
7. Mes do Terremoto
8. Dia do Terremoto
0. Voltar ao Menu Principal
Escolha: 5

--- Análise da Coluna: Impact Score (16944 valores validos) ---
Coeficiente de Assimetria: 7.2335
Coeficiente de Curtose: 89.2287 (Curtose de Excesso: 86.2287)
Desvio Padrão: 139.5928
Variancia: 19486.1582

-----
Medidas de Centralidade Calculadas:
Média: 453.5247
Mediana: 416.0000
Moda: 385.0000

Deseja realizar outra analise estatistica? (s/n): █

```

3.3. Previsão de Impact Score com Base na Magnitude

Essa função é uma previsão de tendências que usa um modelo e métricas obtidas através da análise do dataset, que pode ser verificada na seção 1.1.6 deste relatório, a partir desse modelo o sistema permite que o usuário forneça uma magnitude estimada e receba o valor da previsão de impact score.

```

===== Previsao de Tendencias (Modelo de Regressao Pre-calculado) =====
Usando modelo e metricas fornecidas externamente (ex: Python).

--- Detalhes do Modelo Fornecido ---
Formula: Impact Score = 243.7911 * Magnitude + -845.0397
Intercepto (b0): -845.0397
Inclinacao (b1): 243.7911
Coeficiente de Determinacao (R²): 0.5157
Erro Quadratico Medio (MSE): 9120.2211
Interpretacao R²: O modelo explica aproximadamente 51.5700% da variabilidade no Impact Score (ajuste moderado).
Interpretacao Inclinacao (b1): Em media, para cada aumento de 1 unidade na magnitude, o impact score muda em 243.7911 unidades, de acordo com este modelo.
(Lembre-se: este modelo e uma representacao dos dados analisados externamente.)

Deseja fazer uma previsao de Impact Score para uma magnitude estimada usando este modelo? (s/n): s
Digite a magnitude estimada: 5.8
Para uma magnitude de 5.8000, o Impact Score previsto (usando o modelo fornecido) e: 568.9487

Deseja fazer uma previsao de Impact Score para uma magnitude estimada usando este modelo? (s/n): █

```

3.4. Alerta de Terremotos

Para o alerta de terremotos é necessário inserir a data, horário, cidade, país e magnitude estimada. Com essas informações, o sistema verifica se o país já tem registros anteriores e, caso tenha, verifica qual a classificação de risco do país. A avaliação de risco para o alerta é dada pela magnitude, seguindo o seguinte critério:

Magnitude < 3.5 É considerada fraca e nenhum alerta é enviado

$3.5 \leq \text{Magnitude} \leq 5.4$ Alerta só é enviado se o país tiver perfil de risco "Baixo Risco" ou não tiver nenhum histórico no dataset. Isso porque considera que um desses países não têm preparo para lidar com terremotos, por não ser algo que acontece com frequência.

Magnitude ≥ 5.5 Sempre envia alerta, independente do país ou histórico.

Na imagem abaixo, mostra uma inserção de terremoto e a análise da necessidade do alerta

```
--- Alerta de Terremoto ---
Data do alerta (AAAA-MM-DD): 2025-06-10
Horario do alerta (UTC HH:MM:SS): 19:19:19
Cidade do alerta: Tokyo
País do alerta: Japan
Magnitude estimada: 5.7

Analisando necessidade de alerta...
Evento: Tokyo, Japan - Mag. Estimada: 5.7
Classificacao de risco do país (Japan): Alto Risco
Magnitude  $\geq 5.5$ : Alerta sera enviado independentemente da classificacao do país.

ALERTA SERA ENVIADO!
Alerta enviado para pessoas em um raio de ate 100km do epicentro.
```

3.5. Classificação de Risco Sísmico

Classificar o risco sísmico de um país é algo complicado, e exige um modelo matemático robusto que leva em consideração além dos dados sísmicos, as condições socioeconômicas e o preparo das regiões. Com o dataset usado para esse sistema não tem todas as informações necessárias, então considerando os seguintes parâmetros: média do impact score, número total de terremotos e média das magnitudes, decidiu-se que fazer uma média de cada seria o ideal.

Para cada parâmetro é atribuído um valor de baixo, médio ou alto, de acordo com as seguintes especificações:

Nível	Média Impact Score	Média Magnitude	Número Total de Terremotos
Baixo (1)	0 a 400	0 a 5	0 a 20
Médio (2)	401 a 700	5.1 a 6	21 a 100
Alto (3)	701 ou mais	6.1 ou mais	101 ou mais

Como mostrado na tabela, “baixo” recebe o valor 1, “médio” recebe valor 2 e “alto” recebe valor 3, a classificação final é dada pela média simples, dos valores dos parâmetros que cada país recebe.

Baixo Risco	Médio Risco	Alto Risco
0 a 1	1.1 a 2	2.1 a 3

No sistema, para ver a classificação de cada país, basta selecionar a opção “Ver classificação de risco”, no menu principal, e decidir se quer listar os países que têm a mesma classificação, ou escolher um país e ver sua classificação.

3.6. Executar Benchmarks

Essa opção permite aplicar testes no sistema para verificar como as estruturas se comportam quando submetidas a certas restrições. Além disso, ela aplica métricas para comparação entre as diferentes condições.

As métricas são tempo total de inserção (*insertion_time_ms*), tempo médio de inserção por elemento (*insertion_time_us_avg_element*), tempo médio para busca de registros existentes (*search_existing_time_us_avg*), tempo médio para busca de inexistentes (*search_non_existent_time_us_avg*), tempo médio de remoção (*removal_time_us_avg*), latência média em operações mistas (*avg_latency_us_mixed_ops*), uso estimado de memória (*memory_usage_bytes*), e para Tabela Hash e Tabela Hash perfeita taxa de colisão (*collision_rate_info*).

Para as buscas inexistentes, o sistema pega um registro existente e faz uma pequena alteração para realizar a busca.

```
=== MENU DE BENCHMARK ===
1 - Benchmarks padrão e restritos
2 - Testes de escalabilidade
3 - Teste: Limite de tamanho máximo
4 - Teste: Simulação de concorrência
5 - Teste: Alta latência de I/O
6 - Teste: Chegada irregular de dados
7 - Teste: Reindexação frequente
8 - Salvar resultados em CSV
0 - Voltar
Opcao: █
```

O teste de escalabilidade aplica uma restrição algorítmica e estrutural, onde simula grandes volumes de dados em entrada contínua, forçando reindexação frequente. Usando Scale25pct, Scale50pct, Scale75pct, Scale100pct (Escala , 25%, 50%, 75%, 100%), cada porcentagem é sobre a capacidade das estruturas. Estes cenários de Scale simulam o estresse crescente à medida que o volume de dados se aproxima do máximo.

O teste: limite de tamanho máximo aplica a restrição de memória, onde o número de registros de cada estrutura de dados foi limitado a 10.000 elementos, para esse teste a função utilizada foi a *MaxSize10000* (Tamanho Máximo 10000).

O teste: simulação de concorrência aplica a restrição de processamento, o sistema é submetido a execução forçada com interrupções, usando *ConcurrencySim* (Simulação de Concorrência) que adiciona pausas periódicas (sleep) nos loops de operação, para simular a troca de contexto entre processos. Mede o impacto da concorrência, como travamentos e sincronização (locks).

O teste: alta latência de I/O aplica uma restrição de latência, que simula alta latência no acesso a banco de dados, usando o *HighLatencyIO* (Alta Latência de I/O), que antes de cada operação principal na estrutura (inserção, busca, remoção), adiciona delays aleatórios com sleep com durações aleatórias. Testa como o sistema lida com uma operação de I/O lenta.

O teste: chegada irregular de dados aplica a restrição de dados, que faz uma simulação onde os dados chegam em intervalos não uniformes usando *IrregularArrival* (Chegada Irregular), para verificar seu desempenho sob demanda flutuante.

O teste: indexação frequente é aplicado somente a Perfect Hashing, usando *run_frequent_reindex_tests*, e simula a inserção de novos dados após a construção inicial da tabela hash perfeita.

O menu também oferece a opção de realizar todos os testes de uma vez, aplicando as métricas e avaliando em condições normais, depois tem a opção de salvar os resultados em formato CSV.

4. RESULTADOS DOS BENCHMARK E DISCUSSÃO

Depois que os benchmarks foram executados, foi usada a opção de gerar o arquivo csv, e separou-se os resultados por métricas, as tabelas completas e gráficos feitos usando python podem ser encontrados nos apêndices A e B respectivamente. A seguir seguem os resultados dos testes e as comparações.

4.1. Tempo total de inserção (ms)

Interpretação (Estrutura com o menor tempo):

- Condições normais: A Lista Dupla Ligada possui um menor tempo de inserção (9ms), pois é uma operação $O(1)$ simples.
- Simulação de concorrência: Embora todos sejam muito mais lentos devido à simulação de concorrência, a Lista Dupla Ligada ainda mantém a menor latência.
- Alta latência (entrada/saída): Às estruturas baseadas em árvores e tabelas hash tiveram desempenho similarmente "melhor" no cenário de alta latência, provavelmente porque a latência de entrada/saída domina o tempo de CPU. A HashTable (226106 ms) é um pouco mais rápida.
- Chegada irregular: A Lista Dupla Ligada foi um pouco mais eficiente.
- Tamanho máximo 10000: A Lista Dupla Ligada mantém sua eficiência superior em inserções.
- Escalabilidade (100%): Lista Dupla Ligada (4 ms).
- Escalabilidade (25%): Lista Dupla Ligada (0 ms) .
- Escalabilidade (50%): Lista Dupla Ligada (2 ms).
- Escalabilidade (75%): Lista Dupla Ligada (4 ms).

Conclusão para Tempo Total de Inserção: A Lista Dupla Ligada é a mais eficiente para inserção na maioria dos cenários, exceto no de alta latência.

4.2. Tempo médio de inserção por elemento (us)

Interpretação (Estrutura com o menor tempo):

- Condições normais: Lista Dupla Ligada (0.531 us).
- Simulação de concorrência: Lista Dupla Ligada (314.861 us).
- Alta latência (entrada/saída): HashTable (13344.311 us).
- Chegada irregular: Lista Dupla Ligada (16248.052 us).
- Tamanho máximo 10000: Lista Dupla Ligada (0.200 us).
- Escalabilidade (100%): Lista Dupla Ligada (0.236 us).
- Escalabilidade (25%): Lista Dupla Ligada (0.000 us).
- Escalabilidade (50%): Lista Dupla Ligada (0.236 us).
- Escalabilidade (75%): Lista Dupla Ligada (0.315 us).

Conclusão para Tempo Médio de Inserção por Elemento: A Lista Dupla Ligada é a melhor em quase todas as métricas, exceto pela de alta latência.

4.3. Tempo médio de busca de dados existentes (us)

Interpretação (Estrutura com o menor tempo):

- Condições normais: Perfect HashTable (0.593 us)
- Simulação de concorrência: Perfect HashTable (309.284 us)
- Alta latência (entrada/saída): Perfect HashTable (12979.620 us)
- Chegada irregular: Perfect HashTable (0.900 us)
- Tamanho máximo 10000: Perfect HashTable (0.700 us)
- Escalabilidade (100%): Perfect HashTable (0.535 us)
- Escalabilidade (25%): Perfect HashTable (0.505 us)
- Escalabilidade (50%): Perfect HashTable (0.655 us)
- Escalabilidade (75%): Perfect HashTable (0.635 us)

Conclusão para Tempo Médio de Busca para dados existentes: A PerfectHashTable é a estrutura mais eficiente.

4.4. Tempo médio de busca de dados inexistentes (us)

Interpretação (Estrutura com o menor tempo):

- Condições normais: Perfect HashTable (0.447 us).
- Simulação de concorrência: Perfect HashTable (308.828 us).
- Alta latência (entrada/saída): Perfect HashTable (13524.360 us), sendo a Lista Dupla Ligada e a KD Tree piores.
- Chegada irregular: Perfect HashTable (0.440 us).
- Tamanho máximo 10000: Perfect HashTable (0.410 us).
- Escalabilidade (100%): Perfect HashTable (0.450 us).
- Escalabilidade (25%): Perfect HashTable (0.520 us).
- Escalabilidade (50%): Perfect HashTable (0.490 us).
- Escalabilidade (75%): Perfect HashTable (0.630 us).

Conclusão para Tempo Médio de Busca para dados inexistentes: A Perfect HashTable é a mais eficiente.

4.5. Tempo médio de remoção (us)

Interpretação (Estrutura com o menor tempo):

- Condições normais: Perfect HashTable (0.357 us).
- Simulação de concorrência: Perfect HashTable (320.916 us).
- Alta latência (entrada/saída): Perfect HashTable (12845.890 us).
- Chegada irregular: Perfect HashTable (0.445 us).
- Tamanho máximo 10000: Perfect HashTable (0.350 us).
- Escalabilidade (100%): Perfect HashTable (0.415 us).
- Escalabilidade (25%): Perfect HashTable (0.450 us).
- Escalabilidade (50%): Perfect HashTable (0.505 us).
- Escalabilidade (75%): Perfect HashTable (0.440 us).

Conclusão para Tempo Médio de Remoção: A PerfectHashTable é a mais eficiente em todos os cenários para operações de remoção.

4.6. Latência média de operações mistas

Interpretação (Estrutura com o menor tempo):

- Condições normais: Lista Dupla Ligada (0.467 us).
- Simulação de concorrência: Lista Dupla Ligada (287.747 us).
- Alta latência (entrada/saída): KDTTree (12968.520 us) - As Hash Tables e AVL/Skip List tiveram desempenho similar. A KDTTree se destacou um pouco (porém devido ao problema de inserção de dados, não se pode confiar nesse resultado).
- Chegada irregular: Perfect HashTable (0.873 us)
- Tamanho máximo 10000: Lista Dupla Ligada (0.520 us)
- Escalabilidade (100%): Lista Dupla Ligada (0.373 us)
- Escalabilidade (25%): Perfect HashTable (0.613 us) - Empata com SkipList.
- Escalabilidade (50%): Lista Dupla Ligada (0.947 us).
- Escalabilidade (75%): Lista Dupla Ligada (0.413 us).

Conclusão para Latência Média de Operações Mistas: A Lista Dupla Ligada se mostra mais eficiente na maioria dos cenários de operações mistas.

4.7. Uso de memória estimado

Interpretação (Estrutura com o menor uso de memória):

- Condições normais: Lista Dupla Ligada (6441376 bytes).
- Simulação de concorrência: Lista Dupla Ligada (6441376 bytes).
- Alta latência (entrada/saída): Lista Dupla Ligada (6764576 bytes).
- Chegada irregular: Lista Dupla Ligada (6764576 bytes).
- Tamanho máximo 10000: Lista Dupla Ligada (3999600 bytes).
- Escalabilidade (100%): Lista Dupla Ligada (6764576 bytes).
- Escalabilidade (25%): Lista Dupla Ligada (1630544 bytes).
- Escalabilidade (50%): Lista Dupla Ligada (3341888 bytes).
- Escalabilidade (75%): Lista Dupla Ligada (5053232 bytes).

Conclusão para Uso de Memória: A DoublyLinkedList é a estrutura mais eficiente em termos de uso de memória em todos os cenários.

4.8. Taxa de colisões

Interpretação

- PHT: Secundárias 0 (design)": Significa que, por design, uma Perfecting Hashing Table não possui colisões nas suas tabelas de hash secundárias. Ou seja, uma vez que um elemento é direcionado para uma tabela secundária, ele terá um local único e direto de armazenamento ali, sem a necessidade de tratamento de colisões (como encadeamento ou sondagem linear) dentro dessa tabela. Isso é o que garante o acesso $O(1)$ no pior caso.
- "Primárias: (Não verificado)": Esta parte indica que a taxa de colisões na tabela de hash primária (a primeira função de hash que direciona os elementos para as tabelas secundárias) não foi verificada ou registrada nos dados apresentados. É natural que possam ocorrer colisões na tabela primária, pois ela é responsável por mapear um conjunto potencialmente grande de chaves para um número menor de "balde" (que, no caso de uma PHT, são os pontos de entrada para as tabelas secundárias). No entanto, o design da PHT garante que, mesmo que haja colisões na primária, a estrutura secundária que é construída será sem colisões internas.

Conclusão: A Perfecting Hashing Table (como indicado por "Secundarias 0 (design)"), garante que não haverá colisões no nível onde os dados são armazenados e acessados diretamente (as tabelas secundárias). Isso é o que a torna "perfeita" para buscas, assegurando um tempo de acesso constante e ideal. A existência de colisões na tabela primária é esperada e faz parte do processo de construção, mas não afeta a garantia de $O(1)$ para operações de busca, uma vez que a PHT é construída.

4.9. Trade-off

- Lista Dupla Ligada: Melhor em tempo de inserção (total e médio) e uso de memória. Também se destaca em latência média de operações mistas. Sua desvantagem é o tempo de busca/acesso, que é linear e muito lento para grandes volumes de dados.
- Perfect HashTable: A melhor em tempo médio de busca (existente e inexistente) e tempo médio de remoção. É ideal para cenários onde a busca e remoção são as operações mais frequentes e críticas.
- HashTable (normal): Apresenta um desempenho muito bom para busca e remoção, e é

competitiva com a Perfect HashTable em cenários de alta latência (onde a latência externa domina).

- AVLTree e SkipList: São estruturas equilibradas que oferecem bons tempos de busca, inserção e remoção logarítmicos, mas não se destacam como as mais eficientes em nenhum cenário específico quando comparadas às extremas (Lista Dupla Ligada para inserção/memória e PerfectHashTable para busca/remoção).
- KDTree: Geralmente é a menos eficiente em termos de tempo para as operações básicas de inserção, busca e remoção, especialmente quando comparada às outras estruturas unidimensionais. Sua especialidade é a busca por vizinhos mais próximos em espaços multidimensionais, o que não foi o foco principal dessas métricas.

A escolha da estrutura mais eficiente depende do tipo de operação que será mais frequente e do cenário de uso. No caso, para o Sistema de Monitoramento de Terremotos Globais não existe uma única melhor estrutura para todas as necessidades. Assim, uma abordagem seria utilizar múltiplas estruturas de dados em conjunto, cada uma otimizada para um tipo diferente de operação ou consulta:

- Para Inserção Direta e Armazenamento Primário (se os dados não forem imediatamente indexados): a Lista Duplamente Ligada poderia ser usada para receber os dados brutos do CSV rapidamente.
- Para Consultas Rápidas por Chave Única: HashTable (Normal) já que ela oferece um equilíbrio entre desempenho de busca/inserção/remoção e facilidade de uso para dados dinâmicos. A Perfect HashTable poderia ser considerada se os dados forem muito estáticos e o desempenho de busca for absolutamente crítico, mas a complexidade é maior.
- Para Consultas Geoespaciais: KD-Tree é essencial encontrar terremotos próximos a um ponto ou dentro de uma região, por isso, deveria ser usada especificamente para essas funcionalidades.

Limitações encontradas

Possíveis melhorias e trabalhos futuros

5. CONCLUSÃO

Portanto, foi realizada a implementação de um sistema de classificação e alerta de terremotos com base em um dataset real. O projeto inclui análise estatística, modelagem preditiva e construção e comparação de diferentes estruturas de dados, como listas duplamente encadeadas, árvores AVL, tabelas hash (simples e perfeita), skip lists e KD-trees.

Os resultados dos benchmarks mostraram que não há uma única estrutura ideal para todas as situações: a lista duplamente encadeada se destacou em inserções e uso de memória; a tabela hash perfeita apresentou o melhor desempenho em buscas e remoções. Sendo interessante utilizar diferentes estruturas de dados para cada função a fim de obter resultados mais precisos levando em conta os cenários em que elas atuam.

O sistema final funciona como previsto exceto pela falha na leitura de alguns dados da KD Tree. Assim, ele cumpriu com a proposta de resolver um problema real.

REFERÊNCIAS

GAURAVKUMAR2525. Global Earthquake Dataset (2015-2025). Kaggle, [s.d.]. Disponível em: <<https://www.kaggle.com/datasets/gauravkumar2525/global-earthquake-dataset-2015-2025>>. Acesso em: 05 abr. 2025.

UNITED STATES GEOLOGICAL SURVEY (USGS). Modified Mercalli Intensity Scale. Disponível em: <<https://www.usgs.gov/programs/earthquake-hazards/modified-mercalli-intensity-scale>>. Acesso em: 10 mai. 2025.

ENCYCLOPÆDIA BRITANNICA. Richter scale. Disponível em: <<https://www.britannica.com/science/Richter-scale>>. Acesso em: 08 mai. 2025.

APÊNDICE A - TABELAS DE RESULTADOS DOS BENCHMARK

Observação:

- **Elementos Inseridos:** Quantidade de dados que foram adicionados à estrutura para que ela pudesse ser testada.
- **Elementos Ops:** Quantidade de vezes que alguma operação (como buscar um elemento, remover um elemento etc) foi executada.

TEMPO DE INSERÇÃO (ms)

Árvore AVL

Teste	Valor (ms)	Elementos Inseridos	Elementos Ops
Condições normais	124	16944	1000
Simulação de concorrência	5393	16944	1000
Alta latência (entrada/saída)	229965	16944	200
Chegada irregular	276714	16944	200
Tamanho Máximo 10000	67	10000	100
Escalabilidade (100%)	115	16944	200
Escalabilidade (25%)	29	4236	200
Escalabilidade (50%)	61	8472	200
Escalabilidade (75%)	88	12708	200

Lista Dupla Ligada

Teste	Valor (ms)	Elementos Inseridos	Elementos Ops
Condições normais	9	16944	1000
Simulação de concorrência	5335	16944	1000
Alta latência (entrada/saída)	1194180	16944	200

Chegada irregular	275307	16944	200
Tamanho Máximo 10000	2	10000	100
Escalabilidade (100%)	4	16944	200
Escalabilidade (25%)	0	4236	200
Escalabilidade (50%)	2	8472	200
Escalabilidade (75%)	4	12708	200

Tabela Hash

Teste	Valor (ms)	Elementos Inseridos	Elementos Ops
Condições normais	39	16944	1000
Simulação de concorrência	5438	16944	1000
Alta latência (entrada/saída)	226106	16944	200
Chegada irregular	275807	16944	200
Tamanho Máximo 10000	21	10000	100
Escalabilidade (100%)	36	16944	200
Escalabilidade (25%)	6	4236	200
Escalabilidade (50%)	19	8472	200
Escalabilidade (75%)	26	12708	200

KD Tree

Teste	Valor (ms)	Elementos Inseridos	Elementos Ops
Condições normais	183	16944	1000
Simulação de concorrência	5382	16944	1000
Alta latência (entrada/saída)	228183	16944	200
Chegada irregular	275446	16944	200

Tamanho Máximo 10000	117	10000	100
Escalabilidade (100%)	175	16944	200
Escalabilidade (25%)	24	4236	200
Escalabilidade (50%)	100	8472	200
Escalabilidade (75%)	92	12708	200

Perfect Hashing

Teste	Valor (ms)	Elementos Inseridos	Elementos Ops
Condições normais	75	16944	1000
Simulação de concorrência	5451	16944	1000
Alta latência (entrada/saída)	226140	16944	200
Chegada irregular	276077	16944	200
Tamanho Máximo 10000	42	10000	100
Escalabilidade (100%)	73	16944	200
Escalabilidade (25%)	22	4236	200
Escalabilidade (50%)	40	8472	200
Escalabilidade (75%)	65	12708	200

Skip List

Teste	Valor (ms)	Elementos Inseridos	Elementos Ops
Condições normais	81	16944	1000
Simulação de concorrência	5448	16944	1000
Alta latência (entrada/saída)	226871	16944	200
Chegada irregular	275812	16944	200

Tamanho Máximo 10000	40	10000	100
Escalabilidade (100%)	70	16944	200
Escalabilidade (25%)	16	4236	200
Escalabilidade (50%)	40	8472	200
Escalabilidade (75%)	66	12708	200

TEMPO MÉDIO DE INSERÇÃO 'POR ELEMENTO (us)

Árvore AVL

Teste	Valor (us)	Elementos Inseridos	Elementos Ops
Condições normais	7.318	16944	1000
Simulação de concorrência	318.284	16944	1000
Alta latência (entrada/saída)	13572.061	16944	200
Chegada irregular	16331.091	16944	200
Tamanho Máximo 10000	6.700	10000	100
Escalabilidade (100%)	6.787	16944	200
Escalabilidade (25%)	6.846	4236	200
Escalabilidade (50%)	7.200	8472	200
Escalabilidade (75%)	6.925	12708	200

Lista Dupla Ligada

Teste	Valor (us)	Elementos Inseridos	Elementos Ops
Condições normais	0.531	16944	1000

Simulação de concorrência	314.861	16944	1000
Alta latência (entrada/saída)	70478.04 5	16944	200
Chegada irregular	16248.05 2	16944	200
Tamanho Máximo 10000	0.200	10000	100
Escalabilidade (100%)	0.236	16944	200
Escalabilidade (25%)	0	4236	200
Escalabilidade (50%)	0.236	8472	200
Escalabilidade (75%)	0.315	12708	200

Tabela Hash

Teste	Valor (us)	Elementos Inseridos	Elementos Ops
Condições normais	2.302	16944	1000
Simulação de concorrência	320.940	16944	1000
Alta latência (entrada/saída)	13344.31 1	16944	200
Chegada irregular	16277.56 1	16944	200
Tamanho Máximo 10000	2.100	10000	100
Escalabilidade (100%)	2.125	16944	200
Escalabilidade (25%)	1.416	4236	200
Escalabilidade (50%)	2.243	8472	200
Escalabilidade (75%)	2.046	12708	200

KD Tree

Teste	Valor (us)	Elementos Inseridos	Elementos Ops
-------	------------	---------------------	---------------

Condições normais	10.800	16944	1000
Simulação de concorrência	317.635	16944	1000
Alta latência (entrada/saída)	13466.89 1	16944	200
Chegada irregular	16256.25 6	16944	200
Tamanho Máximo 10000	11.700	10000	100
Escalabilidade (100%)	10.328	16944	200
Escalabilidade (25%)	5.666	4236	200
Escalabilidade (50%)	11.804	8472	200
Escalabilidade (75%)	7.240	12708	200

Perfect Hashing

Teste	Valor (us)	Elementos Inseridos	Elementos Ops
Condições normais	4.426	16944	1000
Simulação de concorrência	321.464	16944	1000
Alta latência (entrada/saída)	13346.73 5	16944	200
Chegada irregular	16294.63 0	16944	200
Tamanho Máximo 10000	4.200	10000	100
Escalabilidade (100%)	4.308	16944	200
Escalabilidade (25%)	5.194	4236	200
Escalabilidade (50%)	4.721	8472	200
Escalabilidade (75%)	5.115	12708	200

Skip List

Teste	Valor (us)	Elementos Inseridos	Elementos Ops
Condições normais	4.780	16944	1000
Simulação de concorrência	321.579	16944	1000
Alta latência (entrada/saída)	13389.04 9	16944	200
Chegada irregular	16277.85 4	16944	200
Tamanho Máximo 10000	4.000	10000	100
Escalabilidade (100%)	4.131	16944	200
Escalabilidade (25%)	3.777	4236	200
Escalabilidade (50%)	4.721	8472	200
Escalabilidade (75%)	5.194	12708	200

TEMPO MÉDIO DE BUSCA DE DADOS EXISTENTES (us)

Árvore AVL

Teste	Valor (us)	Elementos Inseridos	Elementos Ops
Condições normais	3.558	16944	1000
Simulação de concorrência	311.007	16944	1000
Alta latência (entrada/saída)	13320.44 0	16944	200
Chegada irregular	9.030	16944	200
Tamanho Máximo 10000	3.530	10000	100
Escalabilidade (100%)	3.630	16944	200

Escalabilidade (25%)	4.270	4236	200
Escalabilidade (50%)	3.580	8472	200
Escalabilidade (75%)	4.030	12708	200

Lista Dupla Ligada

Teste	Valor (us)	Elementos Inseridos	Elementos Ops
Condições normais	298.559	16944	1000
Simulação de concorrência	505.563	16944	1000
Alta latência (entrada/saída)	13919.55 0	16944	200
Chegada irregular	710.025	16944	200
Tamanho Máximo 10000	180.740	10000	100
Escalabilidade (100%)	309.595	16944	200
Escalabilidade (25%)	65.525	4236	200
Escalabilidade (50%)	146.375	8472	200
Escalabilidade (75%)	230.580	12708	200

Tabela Hash

Teste	Valor (us)	Elementos Inseridos	Elementos Ops
Condições normais	1.613	16944	1000
Simulação de concorrência	333.442	16944	1000
Alta latência (entrada/saída)	12978.66 0	16944	200
Chegada irregular	2.315	16944	200

Tamanho Máximo 10000	1.060	10000	100
Escalabilidade (100%)	0.655	16944	200
Escalabilidade (25%)	1.400	4236	200
Escalabilidade (50%)	1.535	8472	200
Escalabilidade (75%)	0.885	12708	200

KD Tree

Teste	Valor (us)	Elementos Inseridos	Elementos Ops
Condições normais	573.072	16944	1000
Simulação de concorrência	2379.708	16944	1000
Alta latência (entrada/saída)	14176.26 5	16944	200
Chegada irregular	2406.320	16944	200
Tamanho Máximo 10000	413.920	10000	100
Escalabilidade (100%)	583.120	16944	200
Escalabilidade (25%)	118.885	4236	200
Escalabilidade (50%)	406.745	8472	200
Escalabilidade (75%)	445.325	12708	200

Perfect Hashing

Teste	Valor (us)	Elementos Inseridos	Elementos Ops
Condições normais	0.593	16944	1000
Simulação de concorrência	309.284	16944	1000
Alta latência (entrada/saída)	12979.62 0	16944	200
Chegada irregular	0.900	16944	200

Tamanho Máximo 10000	0.700	10000	100
Escalabilidade (100%)	0.535	16944	200
Escalabilidade (25%)	0.505	4236	200
Escalabilidade (50%)	0.655	8472	200
Escalabilidade (75%)	0.635	12708	200

Skip List

Teste	Valor (us)	Elementos Inseridos	Elementos Ops
Condições normais	6.657	16944	1000
Simulação de concorrência	321.137	16944	1000
Alta latência (entrada/saída)	13320.18 0	16944	200
Chegada irregular	6.425	16944	200
Tamanho Máximo 10000	3.820	10000	100
Escalabilidade (100%)	6.660	16944	200
Escalabilidade (25%)	4.236	4236	200
Escalabilidade (50%)	6.290	8472	200
Escalabilidade (75%)	6.680	12708	200

TEMPO MÉDIO DE BUSCA DE DADOS INEXISTENTES (us)

Árvore AVL

Teste	Valor (us)	Elementos Inseridos	Elementos Ops
Condições normais	8.941	16944	1000
Simulação de concorrência	308.922	16944	1000
Alta latência	12910.73	16944	200

(entrada/saída)	0		
Chegada irregular	8.600	16944	200
Tamanho Máximo 10000	4.110	10000	100
Escalabilidade (100%)	3.680	16944	200
Escalabilidade (25%)	8.030	4236	200
Escalabilidade (50%)	5.985	8472	200
Escalabilidade (75%)	3.805	12708	200

Lista Dupla Ligada

Teste	Valor (us)	Elementos Inseridos	Elementos Ops
Condições normais	208.601	16944	1000
Simulação de concorrência	457.215	16944	1000
Alta latência (entrada/saída)	13.040.040	16944	200
Chegada irregular	725.630	16944	200
Tamanho Máximo 10000	129.630	10000	100
Escalabilidade (100%)	230.980	16944	200
Escalabilidade (25%)	24.335	4236	200
Escalabilidade (50%)	92.545	8472	200
Escalabilidade (75%)	151.545	12708	200

Tabela Hash

Teste	Valor (us)	Elementos Inseridos	Elementos Ops
Condições normais	0.753	16944	1000
Simulação de concorrência	301.546	16944	1000
Alta latência (entrada/saída)	13.542.480	16944	200
Chegada irregular	1.380	16944	200
Tamanho Máximo 10000	0.830	10000	100
Escalabilidade (100%)	0.595	16944	200

Escalabilidade (25%)	0.860	4236	200
Escalabilidade (50%)	1.200	8472	200
Escalabilidade (75%)	0.520	12708	200

KD Tree

Teste	Valor (us)	Elementos Inseridos	Elementos Ops
Condições normais	495.189	16944	1000
Simulação de concorrência	1.929.441	16944	1000
Alta latência (entrada/saída)	14.801.075	16944	200
Chegada irregular	1.742.385	16944	200
Tamanho Máximo 10000	275.680	10000	100
Escalabilidade (100%)	526.090	16944	200
Escalabilidade (25%)	88.970	4236	200
Escalabilidade (50%)	222.180	8472	200
Escalabilidade (75%)	380.100	12708	200

Perfect Hashing

Teste	Valor (us)	Elementos Inseridos	Elementos Ops
Condições normais	0.447	16944	1000
Simulação de concorrência	308.828	16944	1000
Alta latência (entrada/saída)	13.524.360	16944	200
Chegada irregular	0.440	16944	200
Tamanho Máximo 10000	0.410	10000	100
Escalabilidade (100%)	0.450	16944	200
Escalabilidade (25%)	0.520	4236	200

Escalabilidade (50%)	0.490	8472	200
Escalabilidade (75%)	0.630	12708	200

Skip List

Teste	Valor (us)	Elementos Inseridos	Elementos Ops
Condições normais	5.725	16944	1000
Simulação de concorrência	318.911	16944	1000
Alta latência (entrada/saída)	13.454.490	16944	200
Chegada irregular	5.395	16944	200
Tamanho Máximo 10000	3.820	10000	100
Escalabilidade (100%)	5.700	16944	200
Escalabilidade (25%)	4.236	4236	200
Escalabilidade (50%)	5.725	8472	200
Escalabilidade (75%)	7.350	12708	200

TEMPO MÉDIO DE REMOÇÃO (us)

Árvore AVL

Teste	Valor (us)	Elementos Inseridos	Elementos Ops
Condições normais	11.558	16944	1000
Simulação de concorrência	318.233	16944	1000
Alta latência (entrada/saída)	13.695.875	16944	200
Chegada irregular	10.665	16944	200
Tamanho Máximo 10000	5.220	10000	100

Escalabilidade (100%)	4.090	16944	200
Escalabilidade (25%)	6.815	4236	200
Escalabilidade (50%)	4.440	8472	200
Escalabilidade (75%)	7.130	12708	200

Lista Dupla Ligada

Teste	Valor (us)	Elementos Inseridos	Elementos Ops
Condições normais	110.499	16944	1000
Simulação de concorrência	324.304	16944	1000
Alta latência (entrada/saída)	12.875.640	16944	200
Chegada irregular	605.400	16944	200
Tamanho Máximo 10000	72.950	10000	100
Escalabilidade (100%)	120.825	16944	200
Escalabilidade (25%)	20.775	4236	200
Escalabilidade (50%)	42.860	8472	200
Escalabilidade (75%)	74.935	12708	200

Tabela Hash

Teste	Valor (us)	Elementos Inseridos	Elementos Ops
Condições normais	1.099	16944	1000
Simulação de concorrência	343.463	16944	1000
Alta latência (entrada/saída)	12.731.305	16944	200
Chegada irregular	1.945	16944	200
Tamanho Máximo 10000	1.150	10000	100
Escalabilidade (100%)	1.220	16944	200

Escalabilidade (25%)	0.985	4236	200
Escalabilidade (50%)	1.950	8472	200
Escalabilidade (75%)	1.375	12708	200

KD Tree

Teste	Valor (us)	Elementos Inseridos	Elementos Ops
Condições normais	316.535	16944	1000
Simulação de concorrência	1.295.708	16944	1000
Alta latência (entrada/saída)	13.329.170	16944	200
Chegada irregular	1.156.605	16944	200
Tamanho Máximo 10000	180.960	10000	100
Escalabilidade (100%)	332.115	16944	200
Escalabilidade (25%)	83.540	4236	200
Escalabilidade (50%)	150.715	8472	200
Escalabilidade (75%)	263.740	12708	200

Perfect Hashing

Teste	Valor (us)	Elementos Inseridos	Elementos Ops
Condições normais	0.357	16944	1000
Simulação de concorrência	320.916	16944	1000
Alta latência (entrada/saída)	12.845.890	16944	200
Chegada irregular	0.445	16944	200
Tamanho Máximo 10000	0.350	10000	100
Escalabilidade (100%)	0.415	16944	200
Escalabilidade (25%)	0.450	4236	200

Escalabilidade (50%)	0.505	8472	200
Escalabilidade (75%)	0.440	12708	200

Skip List

Teste	Valor (us)	Elementos Inseridos	Elementos Ops
Condições normais	6.625	16944	1000
Simulação de concorrência	326.685	16944	1000
Alta latência (entrada/saída)	13.426.690	16944	200
Chegada irregular	6.840	16944	200
Tamanho Máximo 10000	3.820	10000	100
Escalabilidade (100%)	6.625	16944	200
Escalabilidade (25%)	4.236	4236	200
Escalabilidade (50%)	6.625	8472	200
Escalabilidade (75%)	10.630	12708	200

LATÊNCIA MÉDIA DE OPERAÇÕES MISTAS

Árvore AVL

Teste	Valor (ms)	Elementos Inseridos	Elementos Ops
Condições normais	2.320	16944	1000
Simulação de concorrência	372.207	16944	1000
Alta latência (entrada/saída)	13.353.7 93	16944	200
Chegada irregular	3.907	16944	200

Tamanho Máximo 10000	2.873	10000	100
Escalabilidade (100%)	2.107	16944	200
Escalabilidade (25%)	3.853	4236	200
Escalabilidade (50%)	2.160	8472	200
Escalabilidade (75%)	2.073	12708	200

Lista Dupla Ligada

Teste	Valor (ms)	Elementos Inseridos	Elementos Ops
Condições normais	0.467	16944	1000
Simulação de concorrência	287.747	16944	1000
Alta latência (entrada/saída)	13.962.7 53	16944	200
Chegada irregular	1.593	16944	200
Tamanho Máximo 10000	0.520	10000	100
Escalabilidade (100%)	0.373	16944	200
Escalabilidade (25%)	0.607	4236	200
Escalabilidade (50%)	0.947	8472	200
Escalabilidade (75%)	0.413	12708	200

Tabela Hash

Teste	Valor (ms)	Elementos Inseridos	Elementos Ops
Condições normais	0.493	16944	1000
Simulação de concorrência	313.200	16944	1000
Alta latência (entrada/saída)	14.238.2 00	16944	200
Chegada irregular	1.360	16944	200
Tamanho Máximo 10000	0.733	10000	100

Escalabilidade (100%)	0.453	16944	200
Escalabilidade (25%)	0.860	4236	200
Escalabilidade (50%)	1.040	8472	200
Escalabilidade (75%)	0.733	12708	200

KD Tree

Teste	Valor (ms)	Elementos Inseridos	Elementos Ops
Condições normais	1.827	16944	1000
Simulação de concorrência	324.980	16944	1000
Alta latência (entrada/saída)	12.968.5 20	16944	200
Chegada irregular	6.667	16944	200
Tamanho Máximo 10000	3.787	10000	100
Escalabilidade (100%)	1.827	16944	200
Escalabilidade (25%)	1.967	4236	200
Escalabilidade (50%)	1.773	8472	200
Escalabilidade (75%)	3.133	12708	200

Perfect Hashing

Teste	Valor (ms)	Elementos Inseridos	Elementos Ops
Condições normais	0.493	16944	1000
Simulação de concorrência	311.517	16944	1000
Alta latência (entrada/saída)	13.410.8 80	16944	200
Chegada irregular	0.873	16944	200
Tamanho Máximo 10000	0.767	10000	100

Escalabilidade (100%)	0.590	16944	200
Escalabilidade (25%)	0.613	4236	200
Escalabilidade (50%)	0.727	8472	200
Escalabilidade (75%)	0.710	12708	200

Skip List

Teste	Valor (ms)	Elementos Inseridos	Elementos Ops
Condições normais	0.630	16944	1000
Simulação de concorrência	321.751	16944	1000
Alta latência (entrada/saída)	13.426.6 90	16944	200
Chegada irregular	2.920	16944	200
Tamanho Máximo 10000	0.550	10000	100
Escalabilidade (100%)	0.590	16944	200
Escalabilidade (25%)	0.613	4236	200
Escalabilidade (50%)	3.227	8472	200
Escalabilidade (75%)	2.540	12708	200

USO DE MEMÓRIA ESTIMADO

Árvore AVL

Teste	Valor (bytes)	Elementos Inseridos	Elementos Ops
Condições normais	6.568.516	16944	1000
Simulação de concorrência	6.568.516	16944	1000
Alta latência	6.898.116	16944	200

(entrada/saída)			
Chegada irregular	6.898.116	16944	200
Tamanho Máximo 10000	4.078.388	10000	100
Escalabilidade (100%)	6.898.116	16944	200
Escalabilidade (25%)	1.662.832	4236	200
Escalabilidade (50%)	3.407.652	8472	200
Escalabilidade (75%)	5.152.884	12708	200

Lista Dupla Ligada

Teste	Valor (bytes)	Elementos Inseridos	Elementos Ops
Condições normais	6.441.376	16944	1000
Simulação de concorrência	6.441.376	16944	1000
Alta latência (entrada/saída)	6.764.576	16944	200
Chegada irregular	6.764.576	16944	200
Tamanho Máximo 10000	3.999.600	10000	100
Escalabilidade (100%)	6.764.576	16944	200
Escalabilidade (25%)	1.630.544	4236	200
Escalabilidade (50%)	3.341.888	8472	200
Escalabilidade (75%)	5.053.232	12708	200

Tabela Hash

Teste	Valor (bytes)	Elementos Inseridos	Elementos Ops
Condições normais	7.525.084	16944	1000
Simulação de concorrência	7.525.084	16944	1000
Alta latência (entrada/saída)	7.902.684	16944	200
Chegada irregular	7.902.684	16944	200

Tamanho Máximo 10000	4.672.316	10000	100
Escalabilidade (100%)	7.902.684	16944	200
Escalabilidade (25%)	1.904.992	4236	200
Escalabilidade (50%)	3.903.900	8472	200
Escalabilidade (75%)	5.903.292	12708	200

KD Tree

Teste	Valor (bytes)	Elementos Inseridos	Elementos Ops
Condições normais	6.672.540	16944	1000
Simulação de concorrência	6.677.160	16944	1000
Alta latência (entrada/saída)	7.003.920	16944	200
Chegada irregular	7.003.500	16944	200
Tamanho Máximo 10000	4.144.560	10000	100
Escalabilidade (100%)	7.003.500	16944	200
Escalabilidade (25%)	1.695.960	4236	200
Escalabilidade (50%)	3.464.160	8472	200
Escalabilidade (75%)	5.235.300	12708	200

Perfect Hashing

Teste	Valor (bytes)	Elementos Inseridos	Elementos Ops
Condições normais	7.525.084	16944	1000
Simulação de concorrência	7.525.084	16944	1000
Alta latência (entrada/saída)	7.902.684	16944	200
Chegada irregular	7.902.684	16944	200

Tamanho Máximo 10000	4.672.316	10000	100
Escalabilidade (100%)	7.902.684	16944	200
Escalabilidade (25%)	1.904.992	4236	200
Escalabilidade (50%)	3.903.900	8472	200
Escalabilidade (75%)	5.903.292	12708	200

Skip List

Teste	Valor (bytes)	Elementos Inseridos	Elementos Ops
Condições normais	6.770.284	16944	1000
Simulação de concorrência	6.770.284	16944	1000
Alta latência (entrada/saída)	7.097.960	16944	200
Chegada irregular	7.097.960	16944	200
Tamanho Máximo 10000	4.185.792	10000	100
Escalabilidade (100%)	7.097.960	16944	200
Escalabilidade (25%)	1.662.832	4236	200
Escalabilidade (50%)	3.407.652	8472	200
Escalabilidade (75%)	5.152.884	12708	200

TAXA DE COLISÃO

nota-se "Secundarias 0 (design); Primarias: (Nao verificado)", indicando que colisões secundárias são evitadas por design, mas as colisões primárias não foram verificadas.

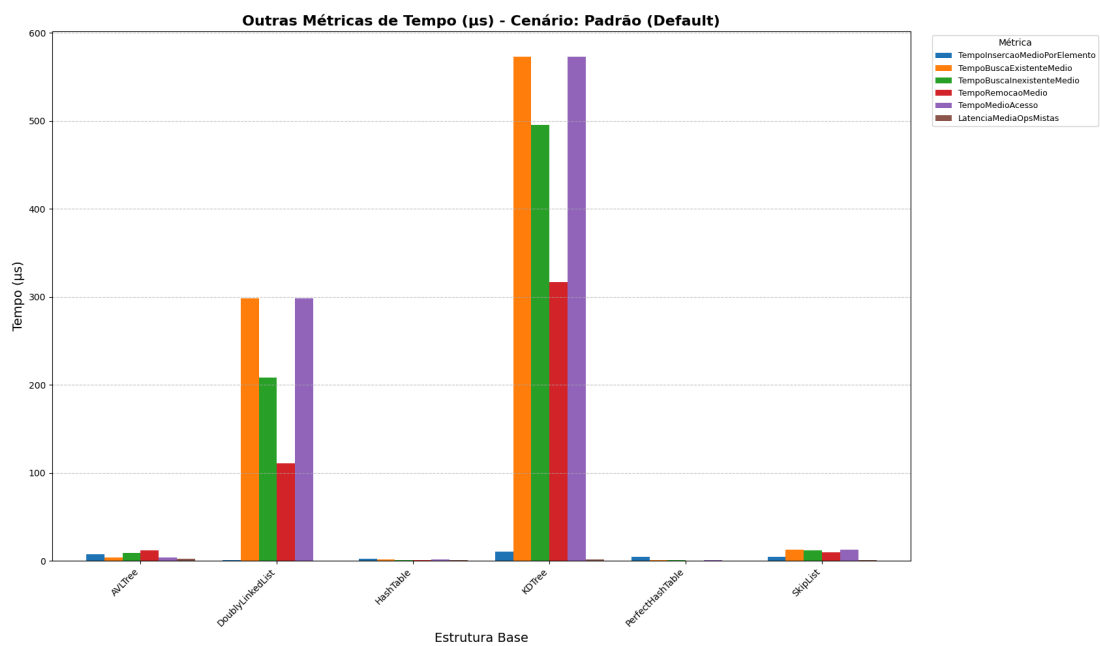
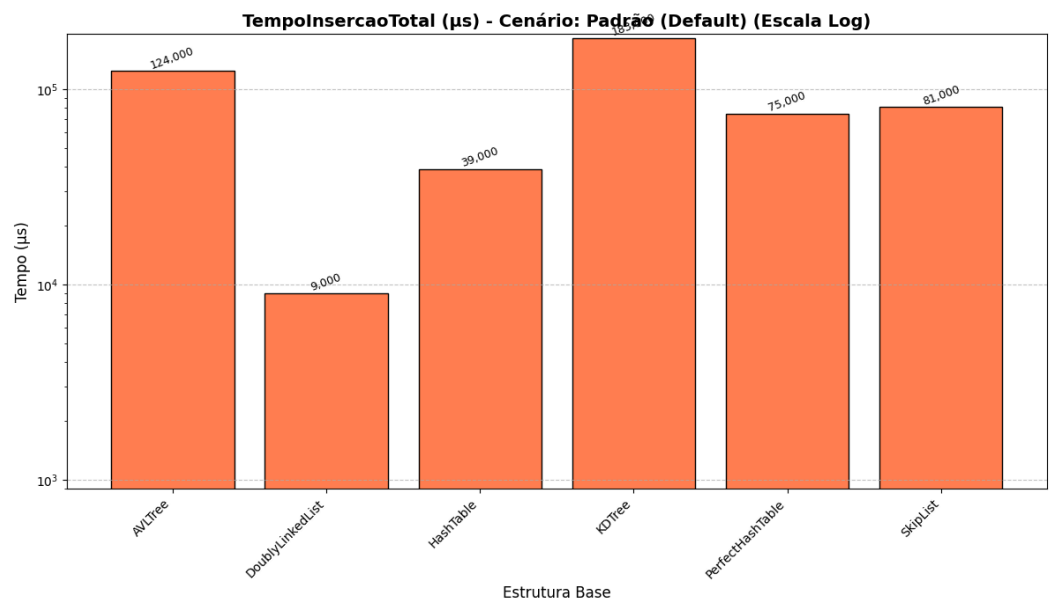
Perfect Hashing

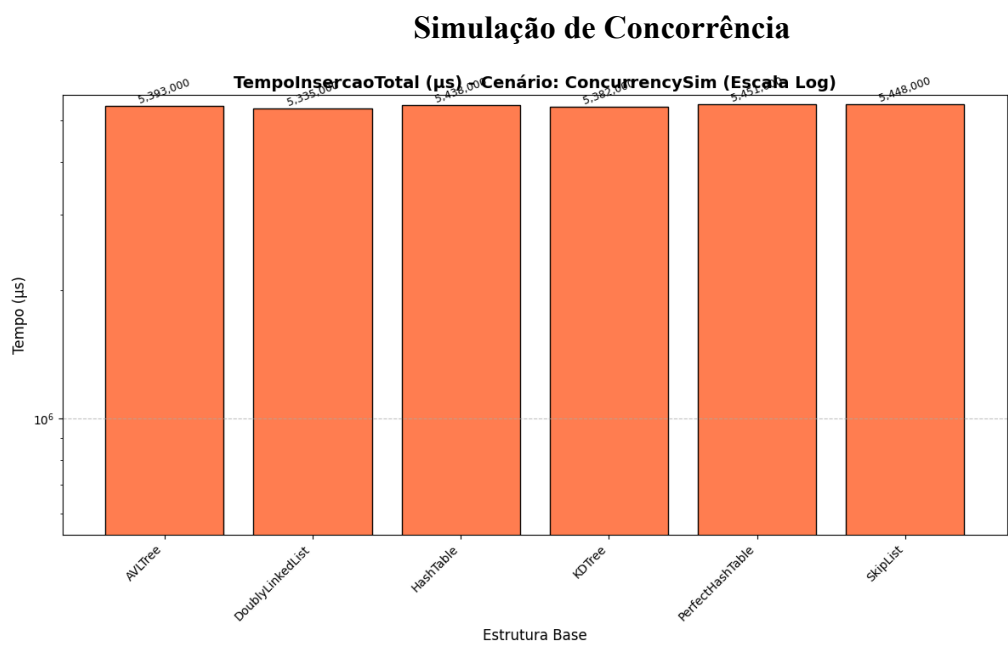
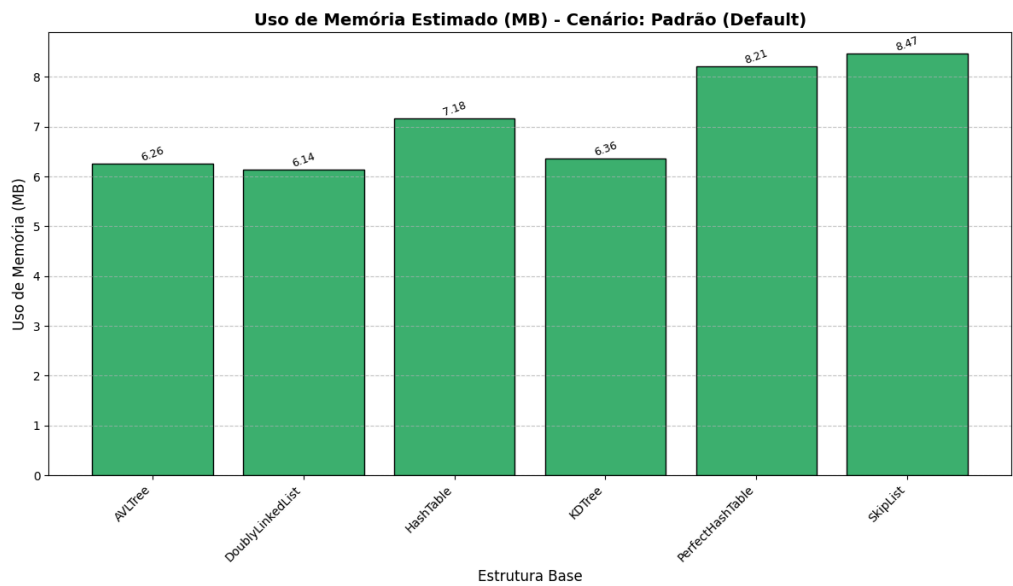
Teste	Valor	Elementos	Elementos Ops
-------	-------	-----------	---------------

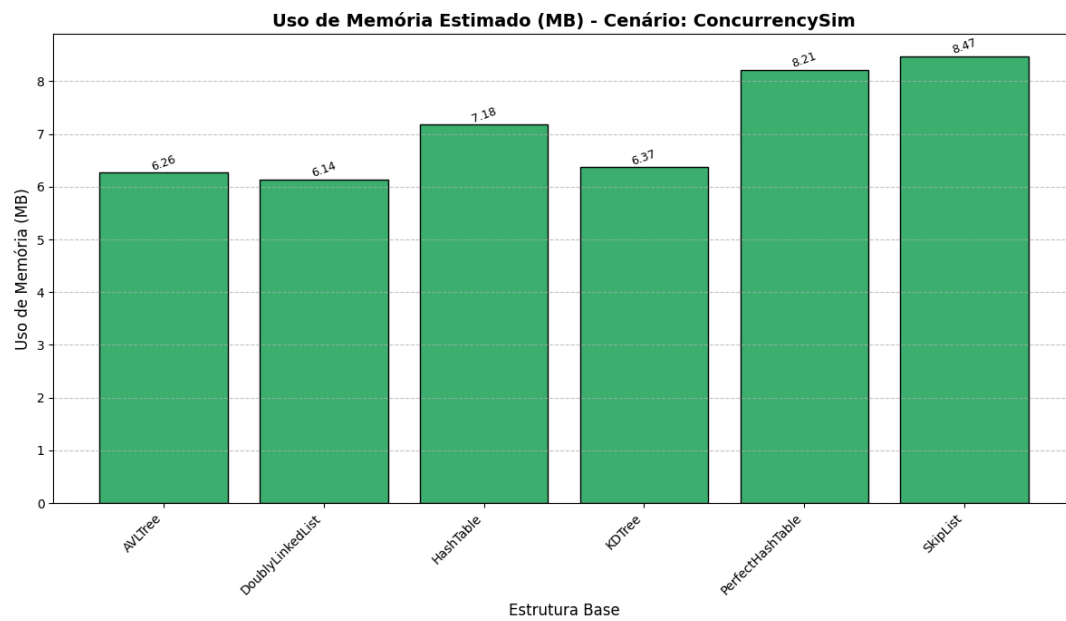
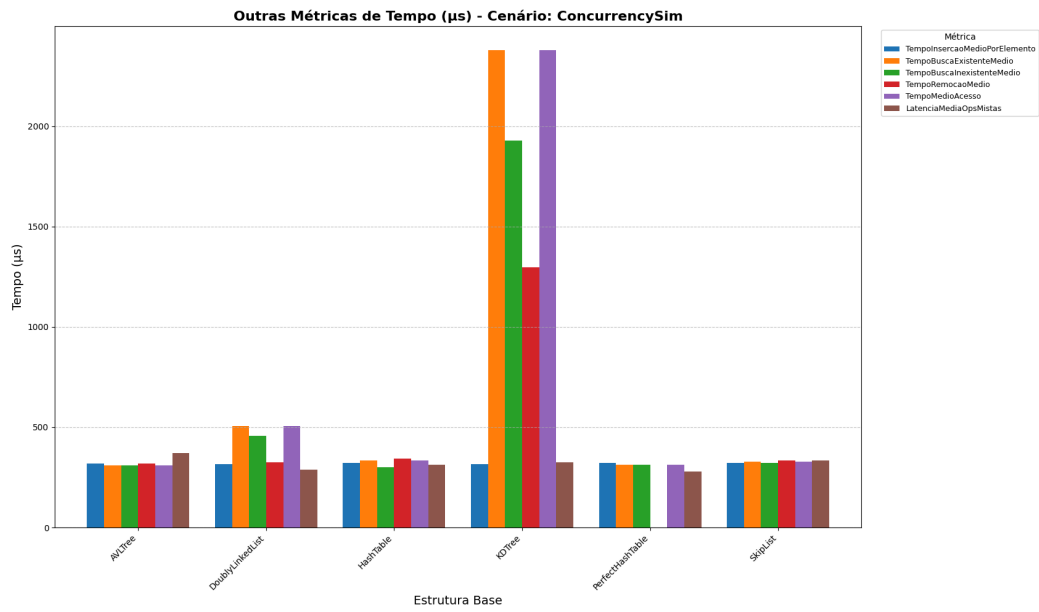
		Inseridos	
Condições normais	PHT: Secundarias 0 (design); Primarias: (Não verificado)	16944	1000
Simulação de concorrência	PHT: Secundarias 0 (design); Primarias: (Não verificado)	16944	1000
Alta latência (entrada/saída)	PHT: Secundarias 0 (design); Primarias: (Não verificado)	16944	200
Chegada irregular	PHT: Secundarias 0 (design); Primarias: (Não verificado)	16944	200
Tamanho Máximo 10000	PHT: Secundarias 0 (design); Primarias: (Não verificado)	10000	100
Escalabilidade (100%)	PHT: Secundarias 0 (design); Primarias: (Não verificado)	16944	200
Escalabilidade (25%)	PHT: Secundarias 0 (design); Primarias: (Não verificado)	4236	200
Escalabilidade (50%)	PHT: Secundarias 0 (design); Primarias: (Não verificado)	8472	200
Escalabilidade (75%)	PHT: Secundarias 0 (design); Primarias: (Não verificado)	12708	200

APÊNDICE B - GRÁFICOS DOS RESULTADOS DOS BENCHMARK

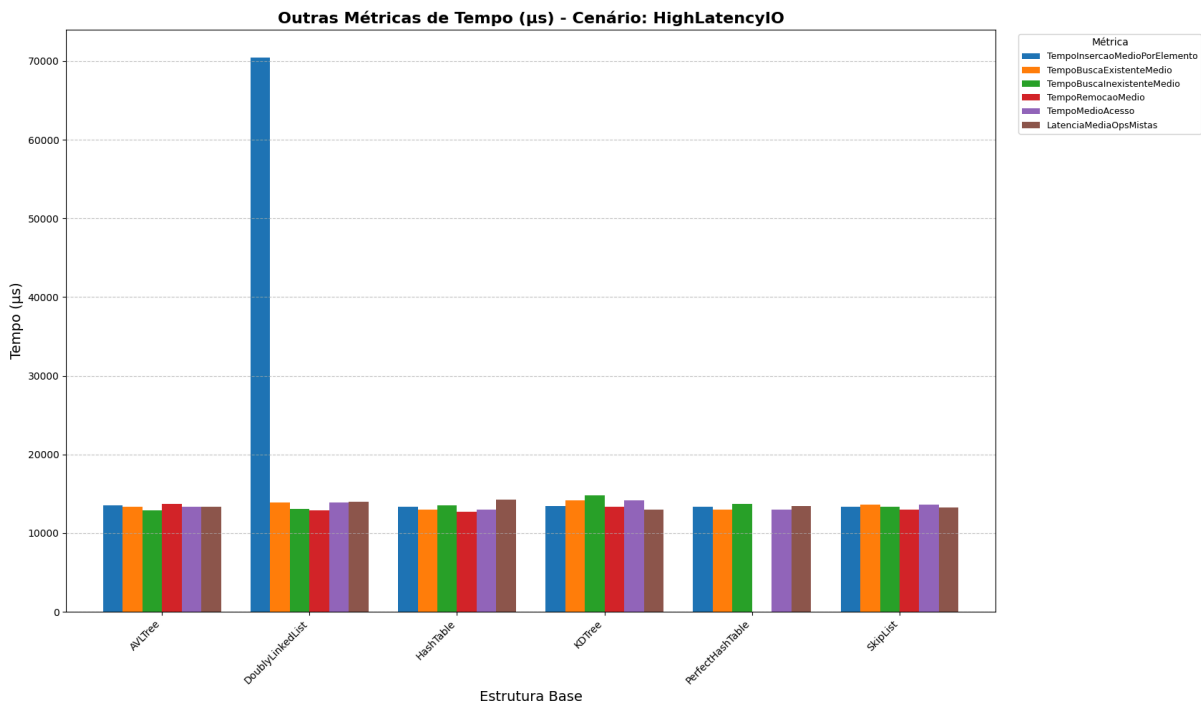
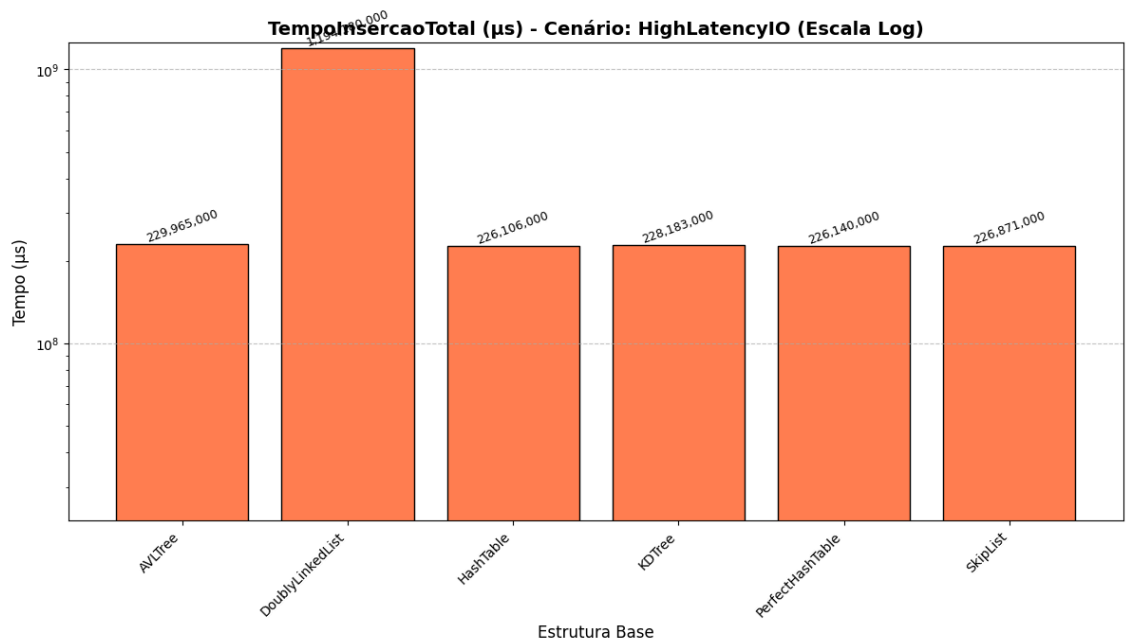
Padrão

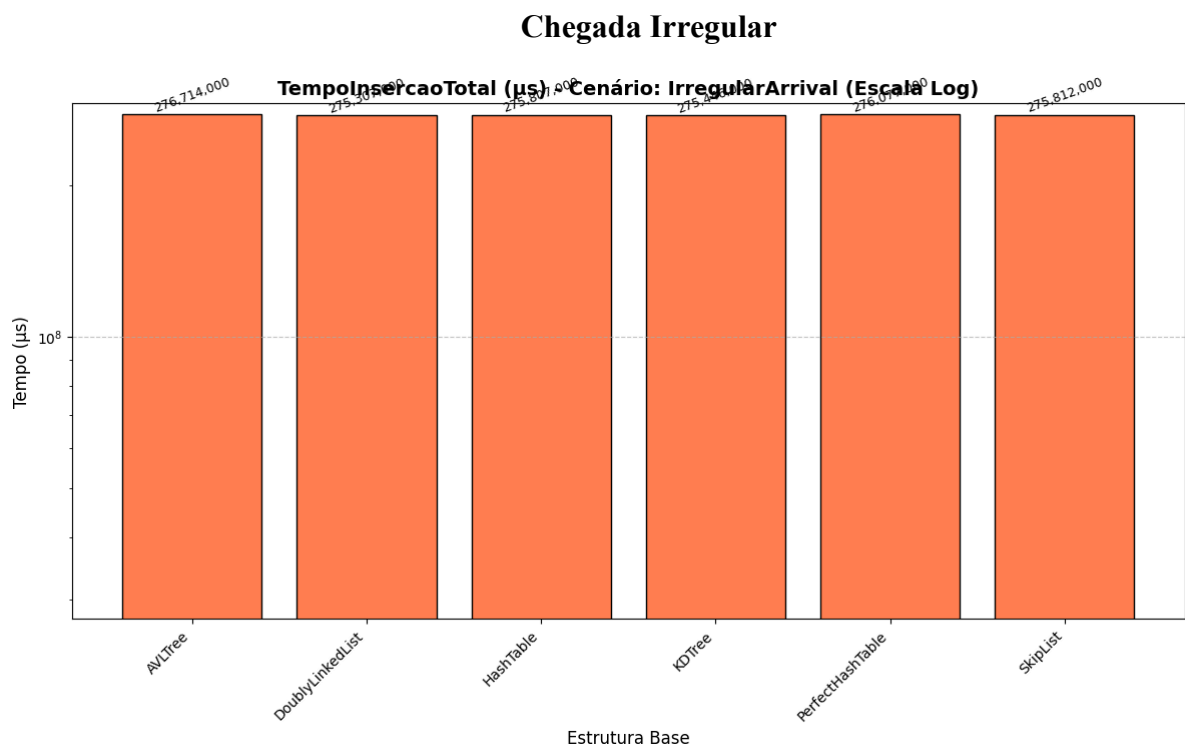
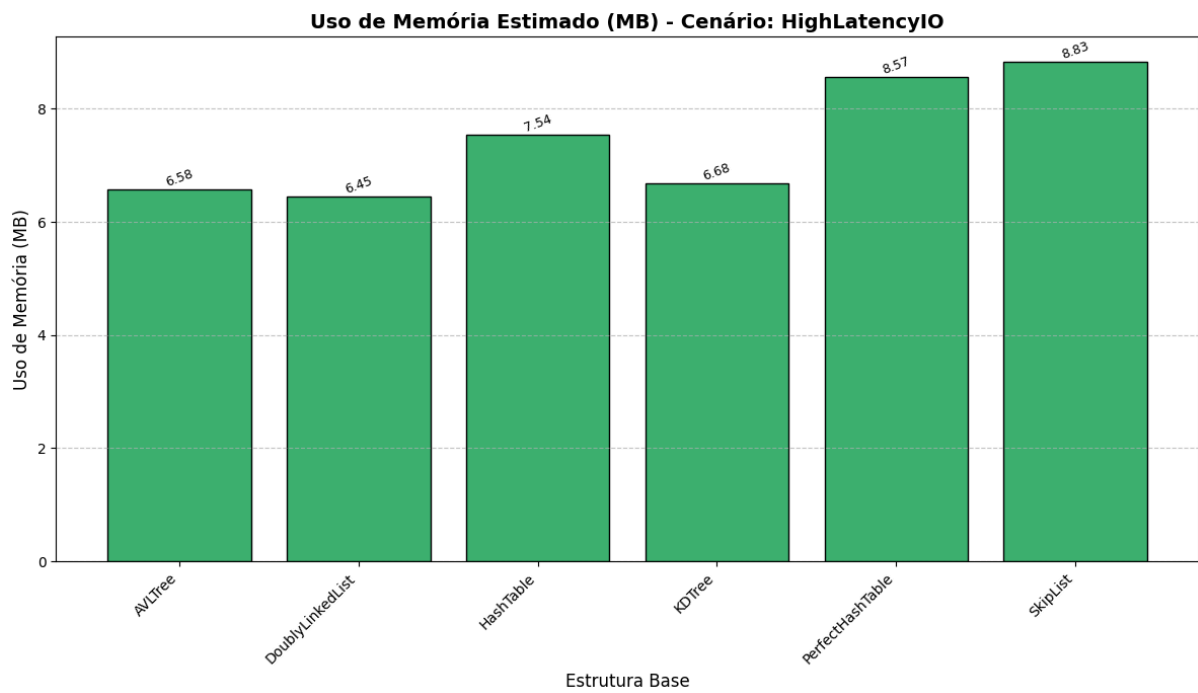


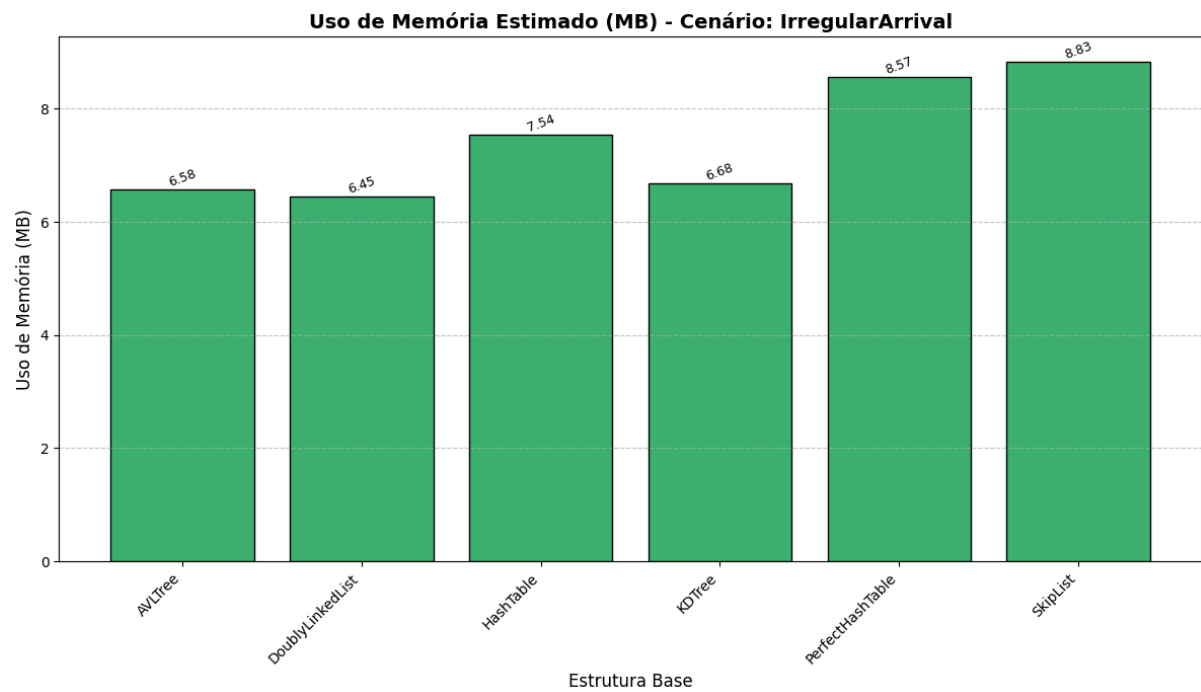
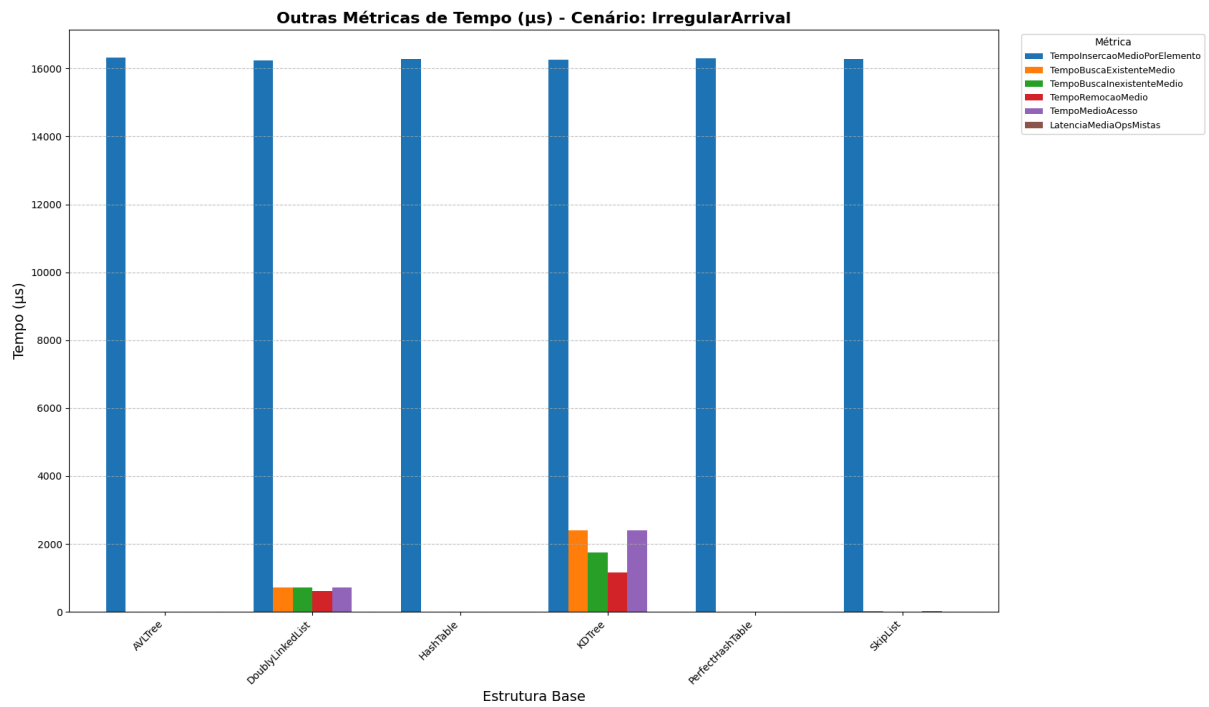




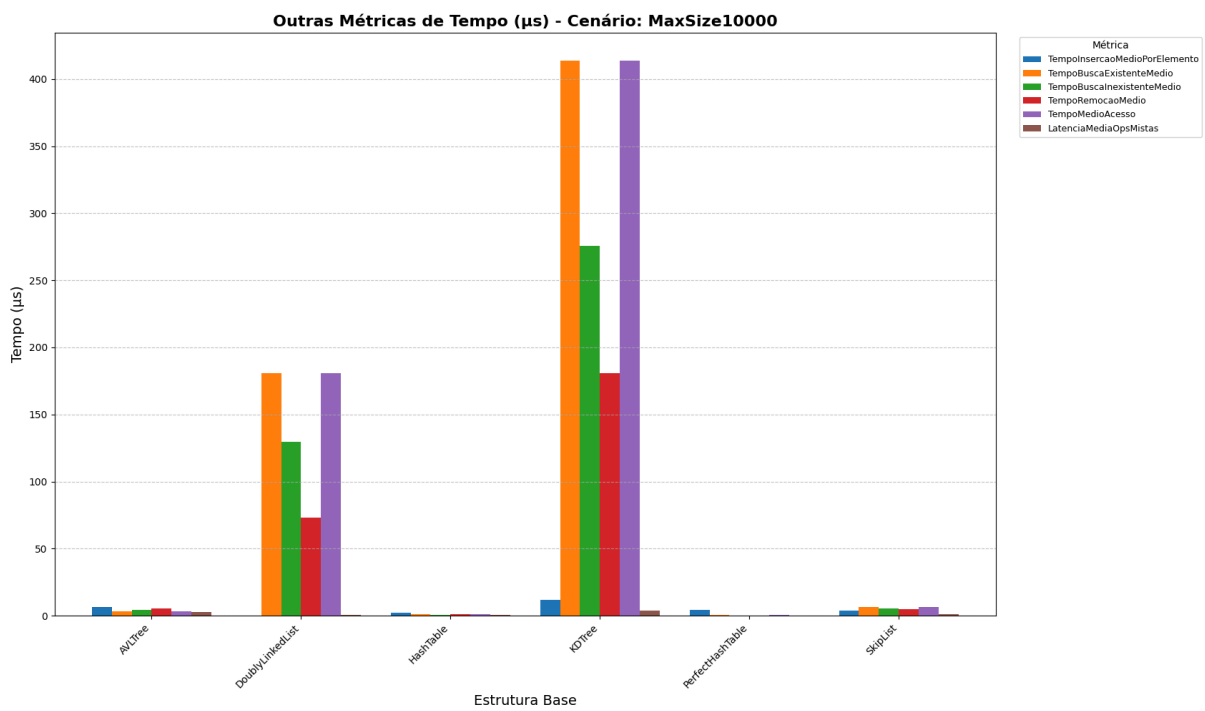
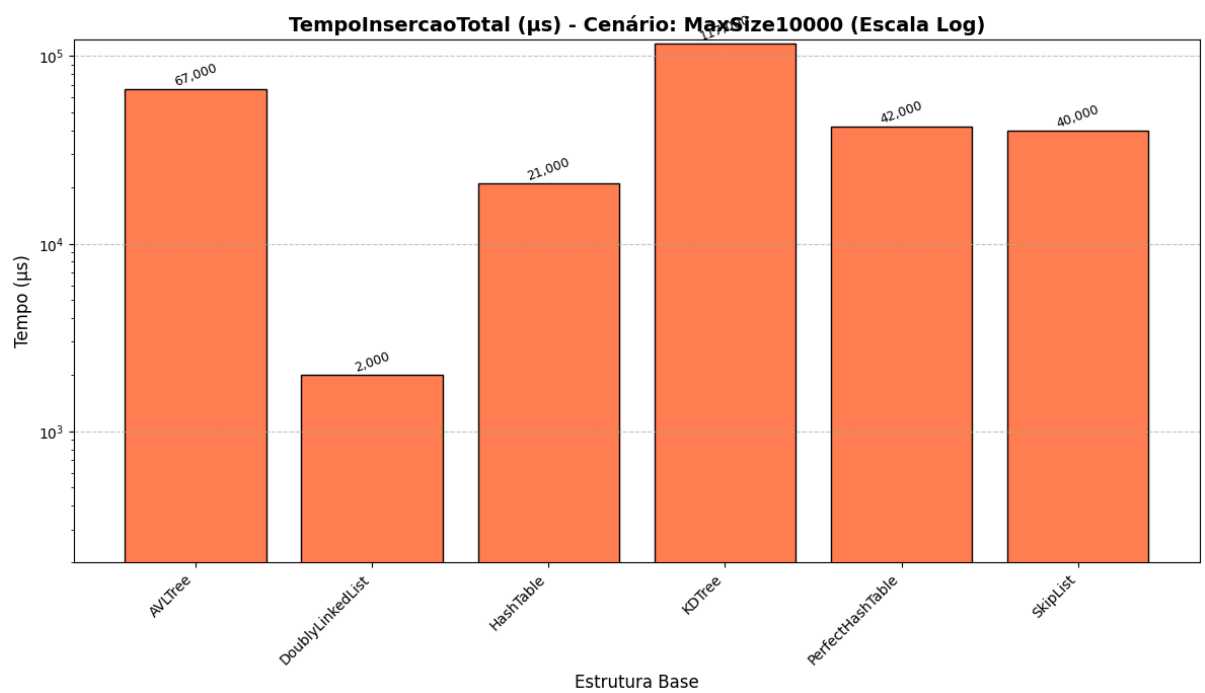
Alta Latência (Entrada/Saída)

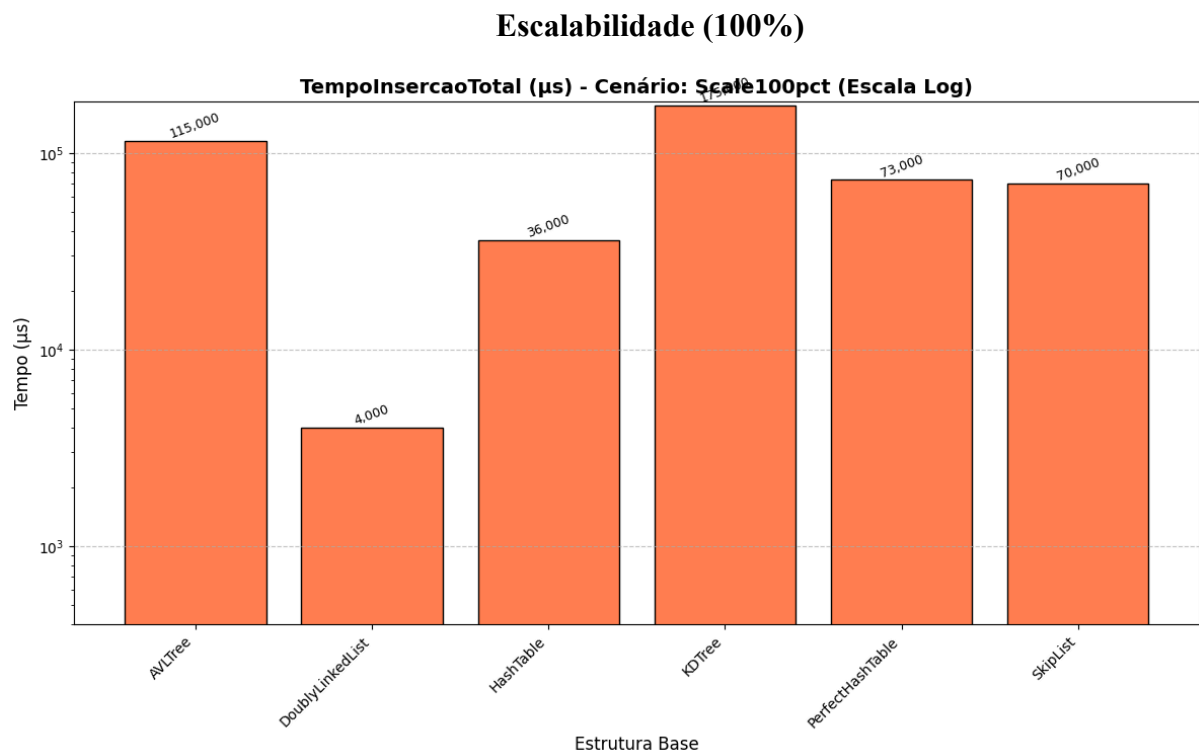
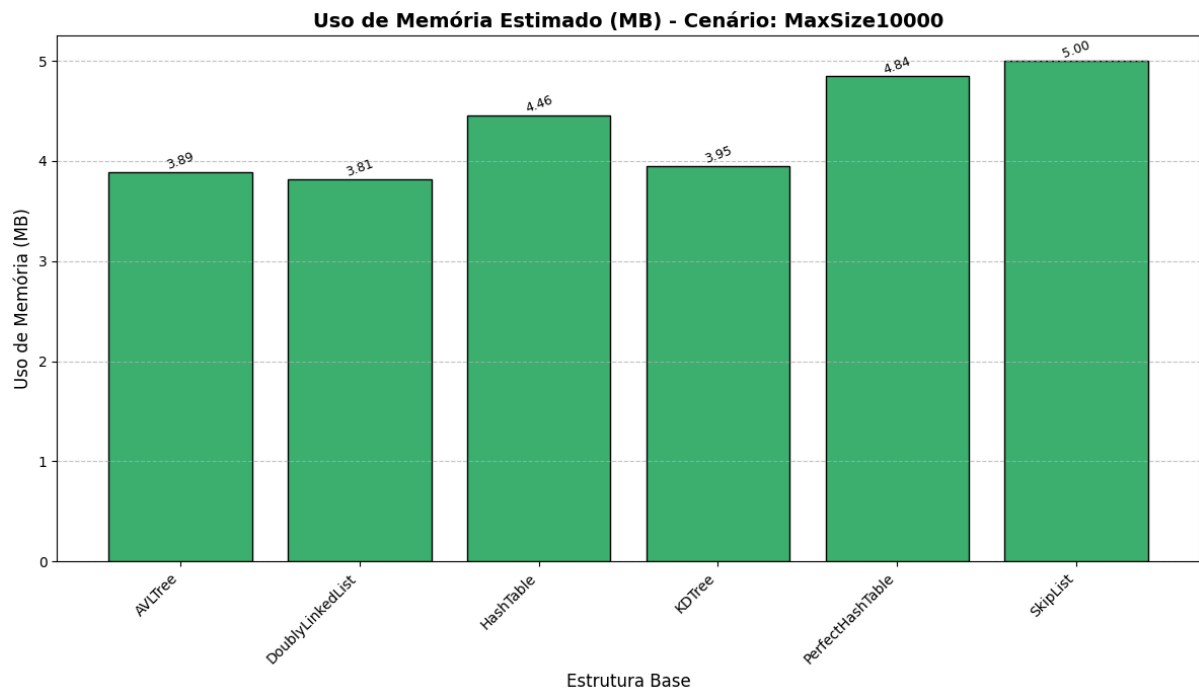


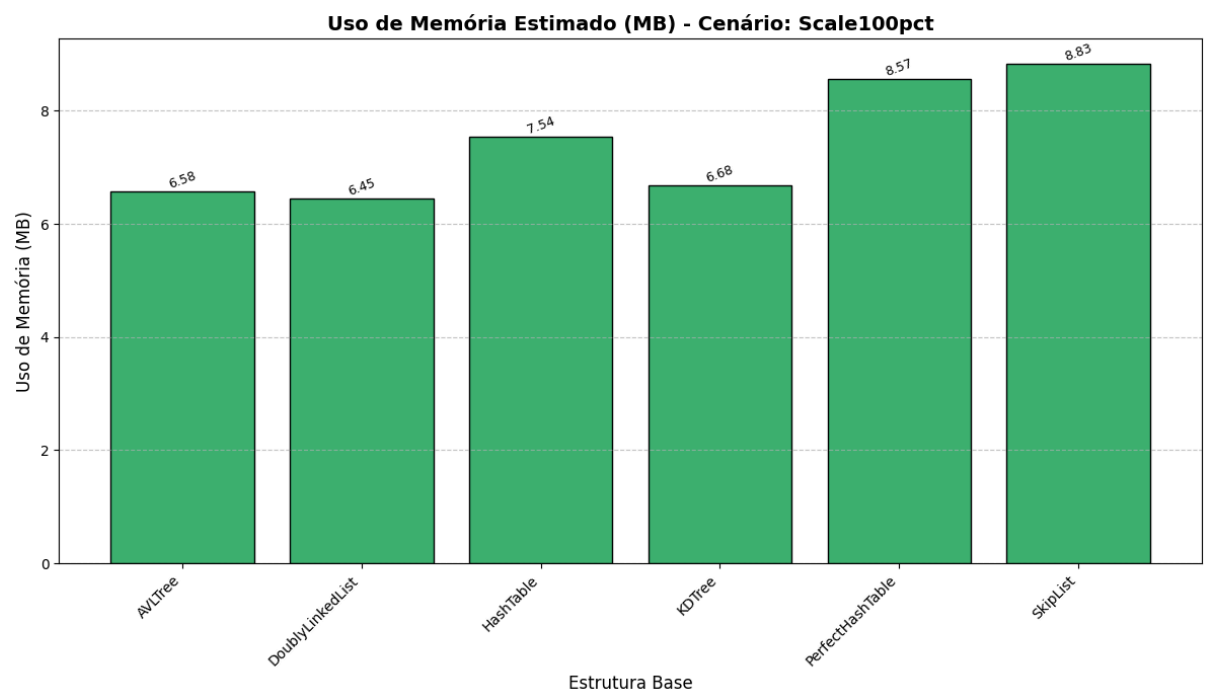
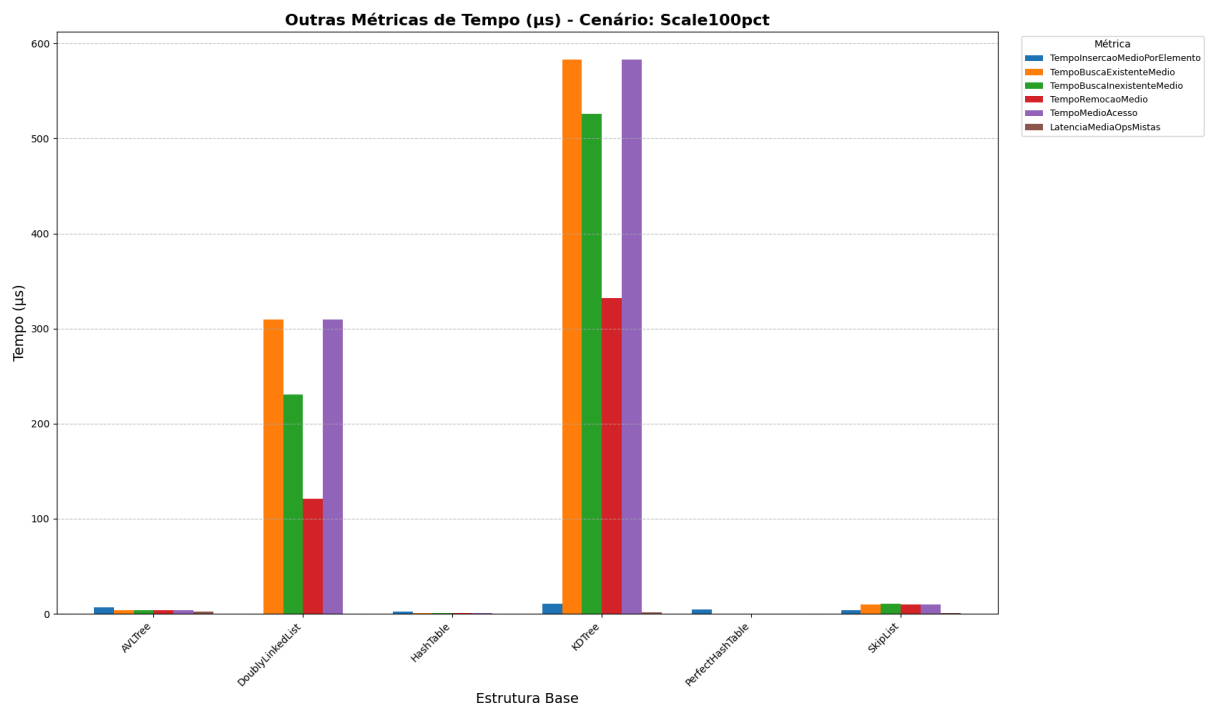




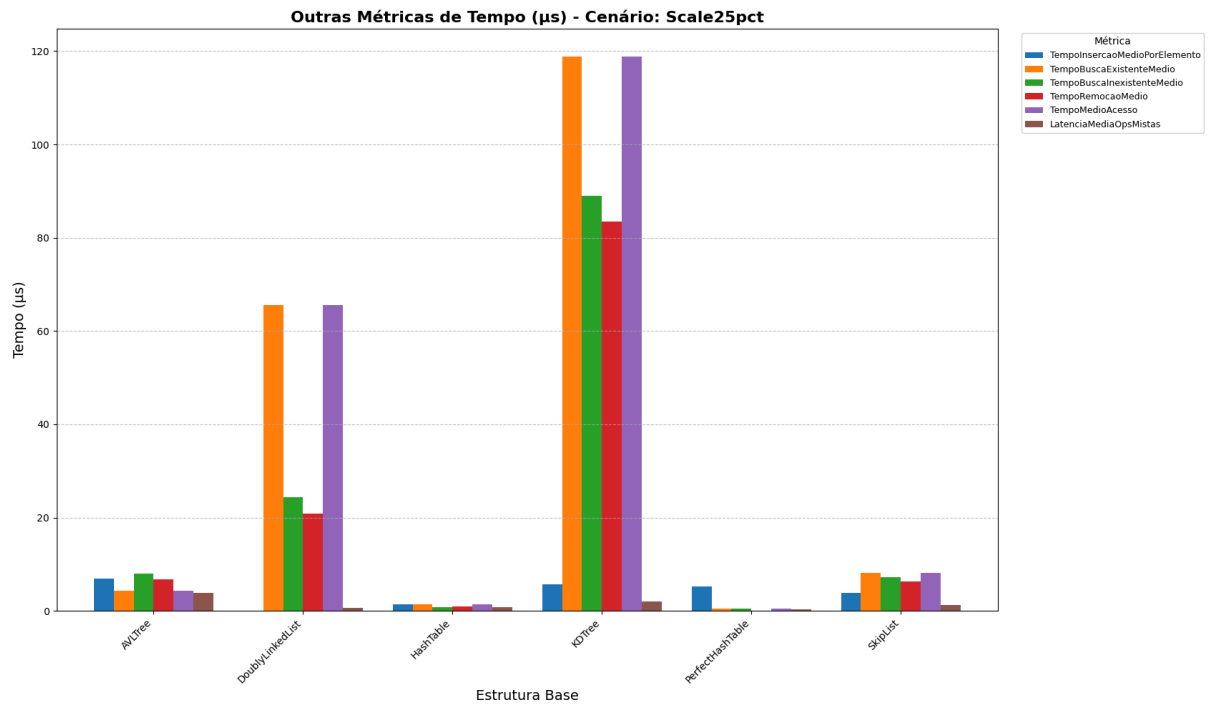
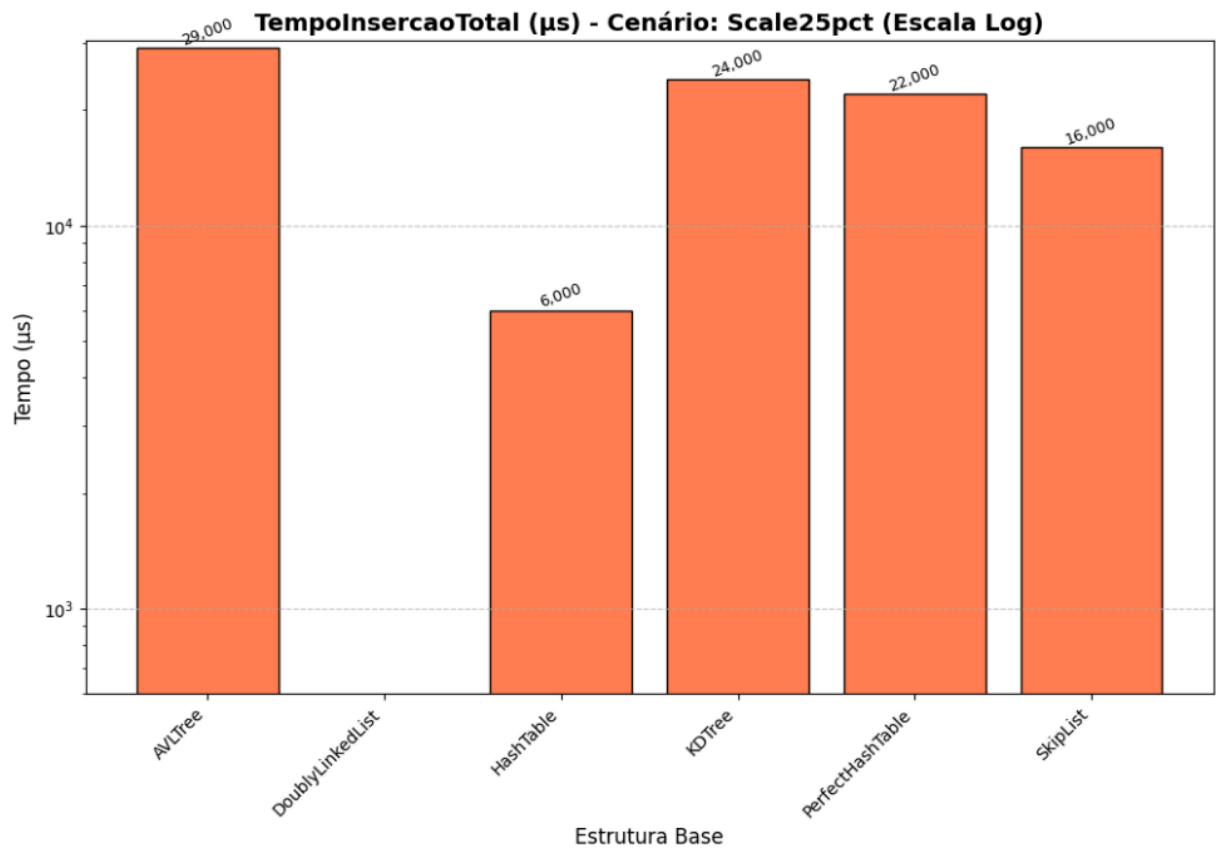
Tamanho Máximo 10000

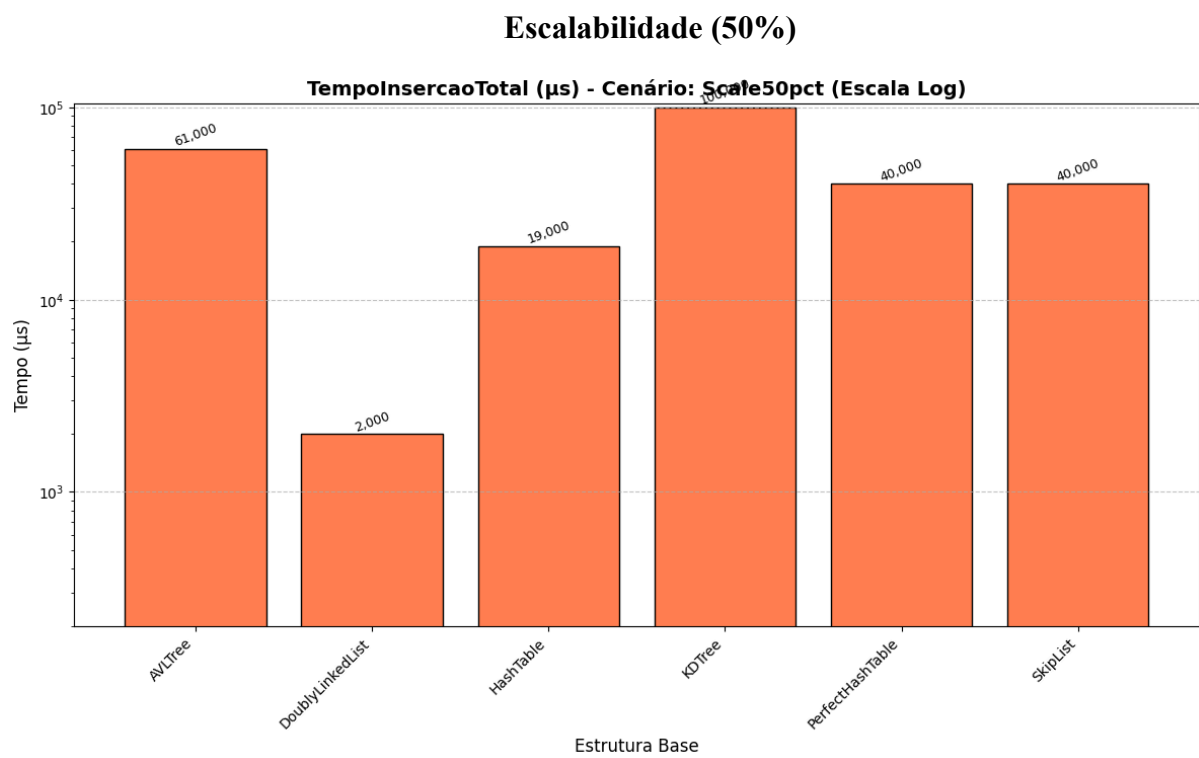
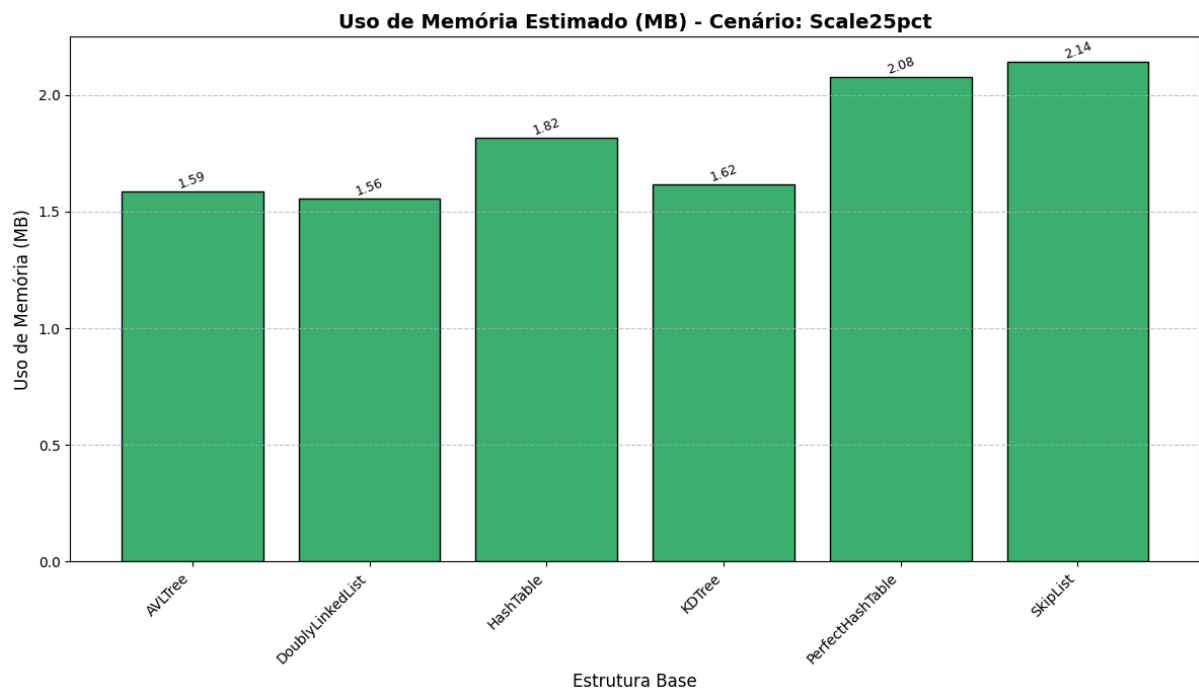


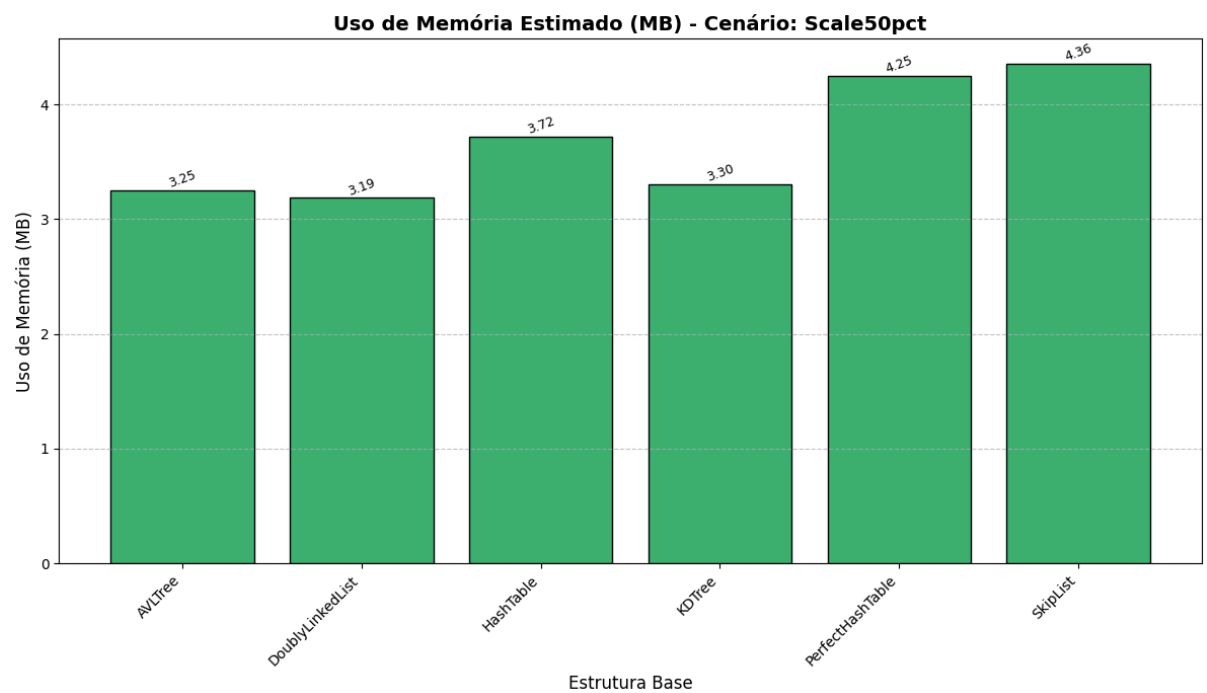
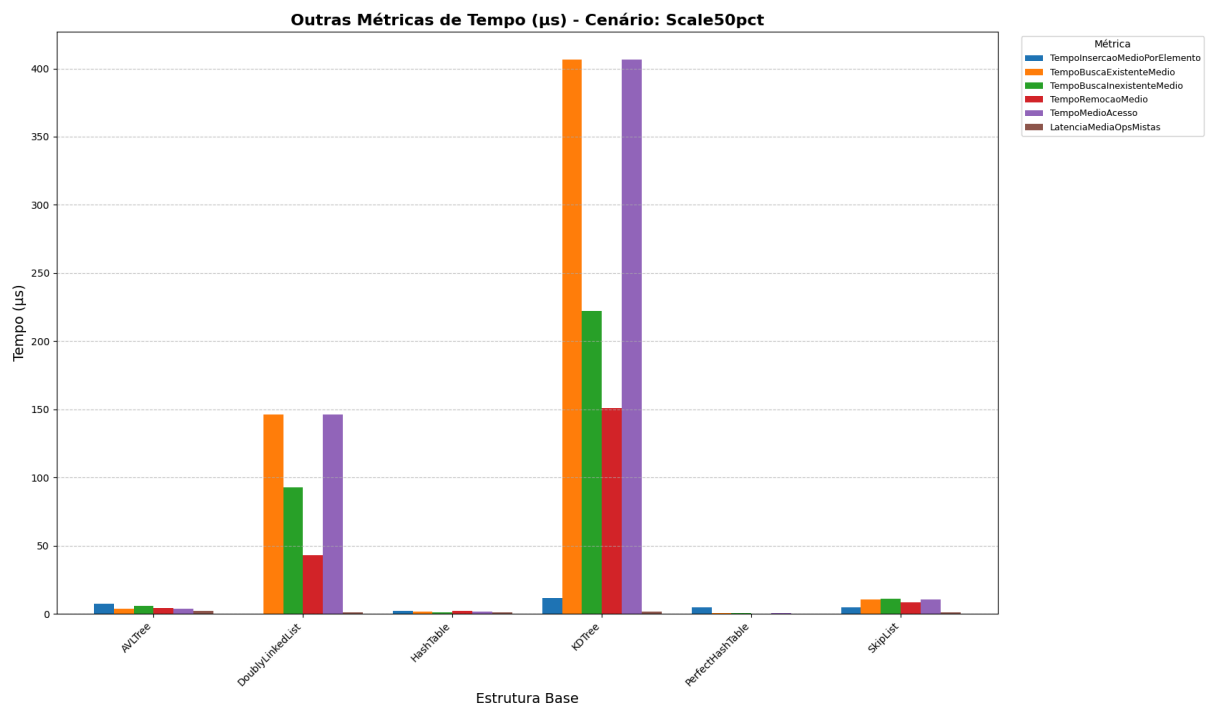




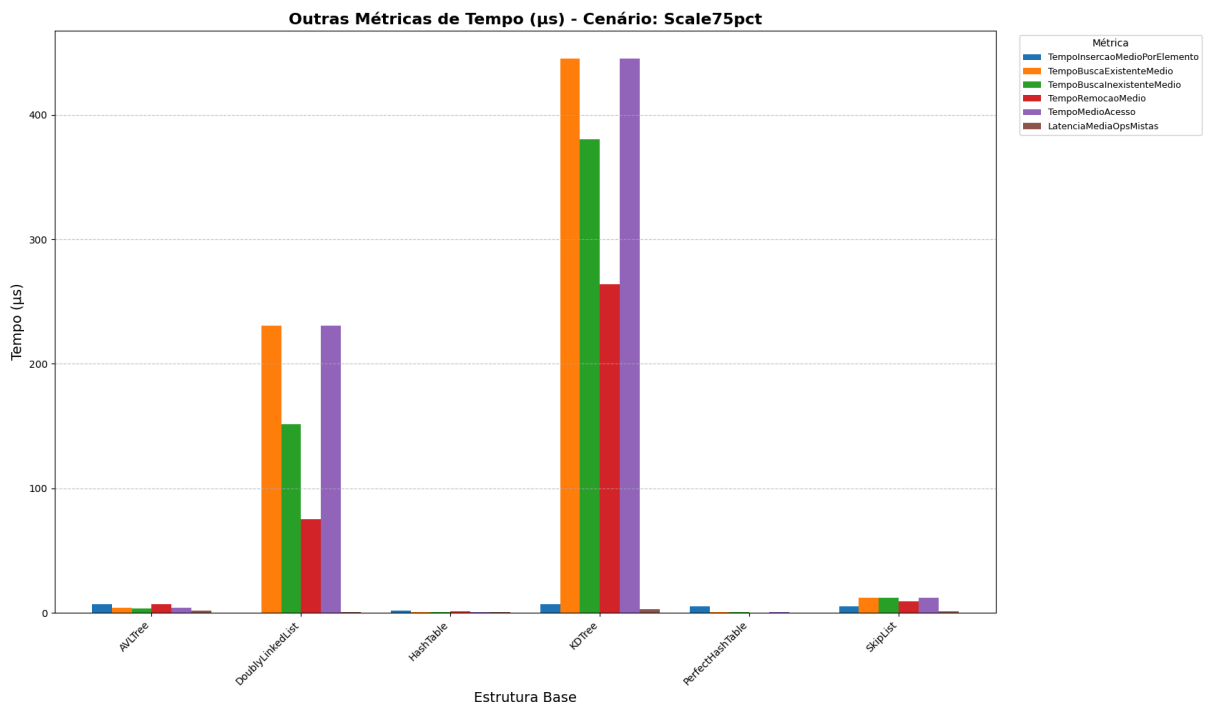
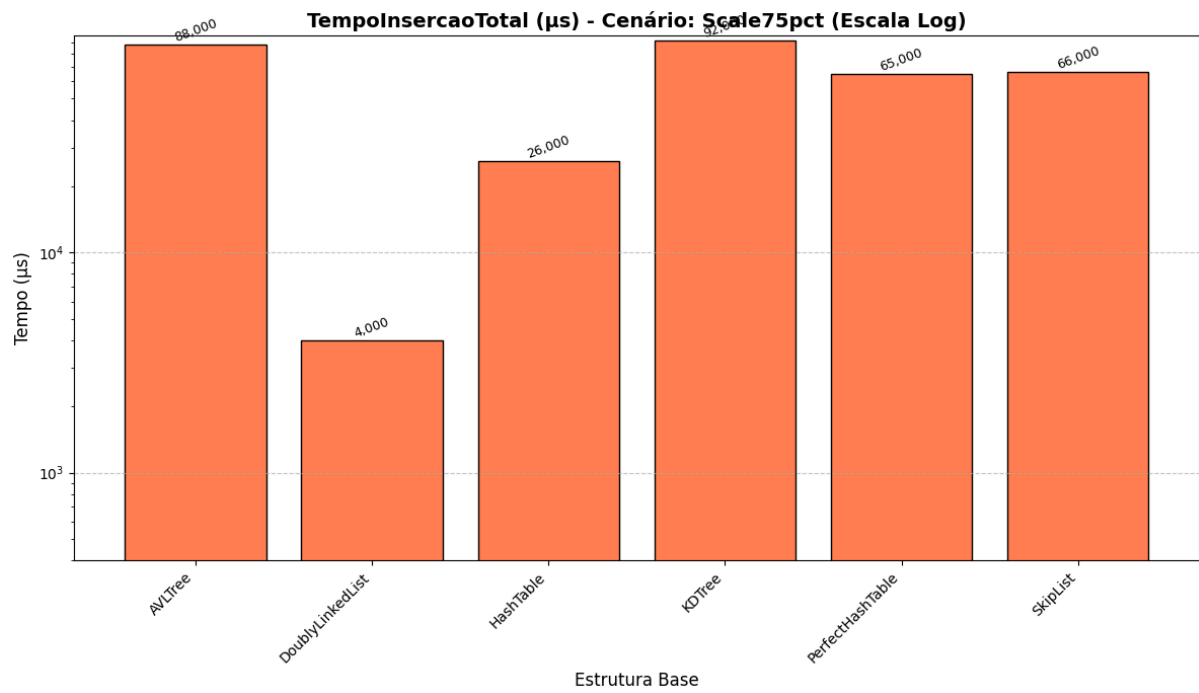
Escalabilidade (25%)

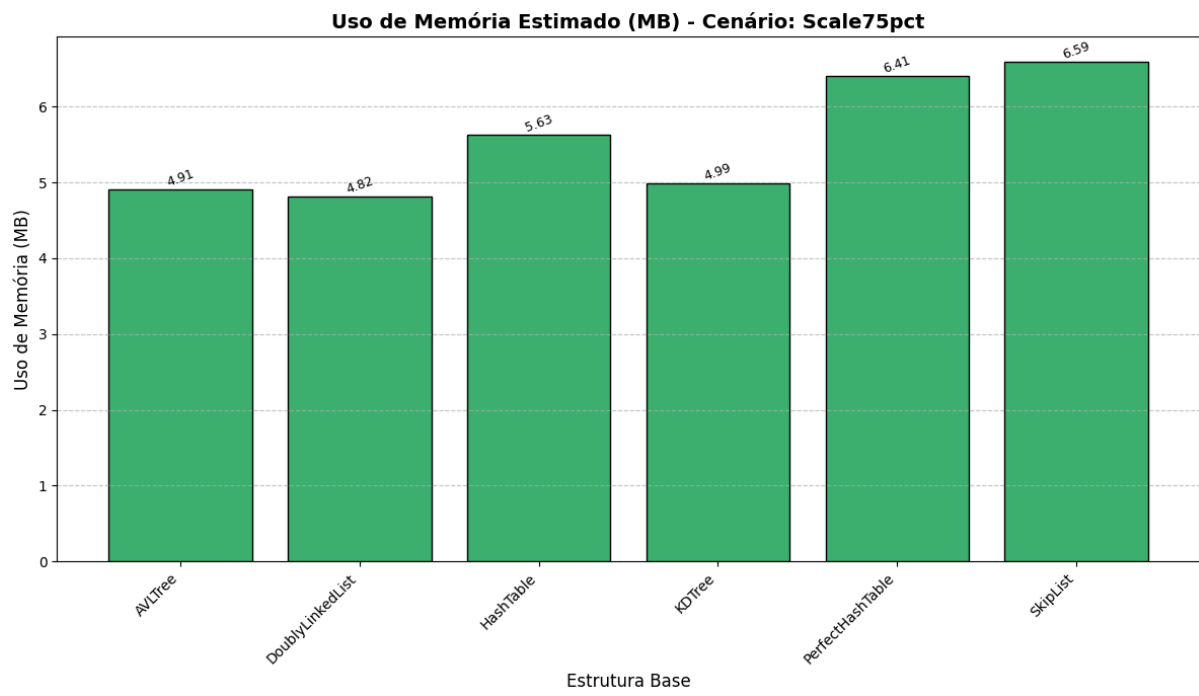






Escalabilidade (75%)





APÊNDICE C - PROMPT USADO PARA APOIO DE INTELIGÊNCIA ARTIFICIAL

Estou fazendo um trabalho de estrutura de dados em C/C++ e preciso ler um dataset sobre terremotos globais, colocá-los em estruturas de dados (no caso lista encadeada, árvore avl, tabela hash, skip list, kd tree) e realizar as operações inserir, remover e buscar.

Precisa ter apenas 1 arquivo executável (main)

csv_reader.cpp

função que irá ler o excel csv

main.cpp

Arquivo executável que terá um menu feito com switch que o usuário selecionará a estrutura que ele quer usar

O menu precisará ser dividido em 2:

- Estrutura de dados
 - Lista dupla ligada
 - Árvore AVL
 - Tabela Hash
 - Skip-List
 - KD-Tree
 - Perfect hashing
- Cálculos estatísticos sobre os dados
- Filtragem e ordenação dos dados para análise otimizada
- Previsão de tendências
- Benchmark

`filtragem.cpp`

função que filtra os dados do dataset removendo as linhas que possuem dados faltando (exceto pela hora)

`benchmark.cpp`

função que fará uma tabela de benchmark entre as estruturas comparando o tempo de inserção, remoção e busca, meça a quantidade de memória alocada e eficiência algorítmica.

`statistic.cpp`

função que fará cálculos estatísticos dos dados numéricos (transformar de string para double):

Média, mediana, desvio padrão e variância de cada variável.

`tendendia.cpp`

Função que realizará a regressão linear para prever o impact score com base na magnitude estimada do terremoto.

`classification.cpp`

obs: dados estão sendo salvos como string, precisa converter

Função que irá dividir os países do dataset em três tipos de risco:

- alto risco
- médio risco
- baixo risco

Com base na conta matemática a seguir:

Critérios de Classificação

Cada região terá um nível de risco atribuído com base em um score composto por:

- Parâmetro 1: Número total de terremotos
 - 101 ou mais- alto
 - 100-21- médio
 - 20 ou menos- baixo
- Parâmetro 2: Média das magnitudes
 - 0 a 5- baixo
 - 5.1 e 6- médio
 - 6.1 ou mais- alto
- Parâmetro 3: Média do *Impact Score*
 - 0 a 400- baixo
 - 401 a 700- médio
 - 701 ou mais- alto

Depois, será atribuído um valor para cada nível de cada parâmetro, sendo:

- Baixo: 1
- Médio: 2
- Alto: 3

Será somado e feito uma média do nível dos parâmetros e o nível do país será definido com base no número calculado, sendo:

- 0 a 1: Baixo
- 1.1 a 2: Médio
- 2.1 a 3: Alto

`lista_dupla_ligada.cpp`

função que irá implementar a lista dupla encadeada a partir do dataset (csv previamente lido), sendo inserir, remover e buscar

`avl_tree.cpp`

função que irá implementar a árvore AVL a partir do dataset (csv previamente lido), sendo inserir, remover e buscar

`hash_table.cpp`

função que irá implementar a tabela hash a partir do dataset (csv previamente lido), sendo inserir, remover e buscar

`skip_list.cpp`

função que irá implementar a skip list a partir do dataset (csv previamente lido), sendo inserir, remover e buscar

`KD_tree.cpp`

função que irá implementar a kd tree a partir do dataset (csv previamente lido), sendo inserir, remover e buscar

`perfect_hash.cpp`

A PerfectHashTable que você pediu e a HashTable original que você tinha (que usa encadeamento para resolver colisões) diferem fundamentalmente na forma como lidam com colisões de hash e, conseqüentemente, no desempenho de busca

`operations_menu.h`

header que implementa um submenu interativo para o usuário gerenciar dados de terremotos (possui as funções de inserir, remover, buscar e listar)